

기초 수학 & 지도 학습

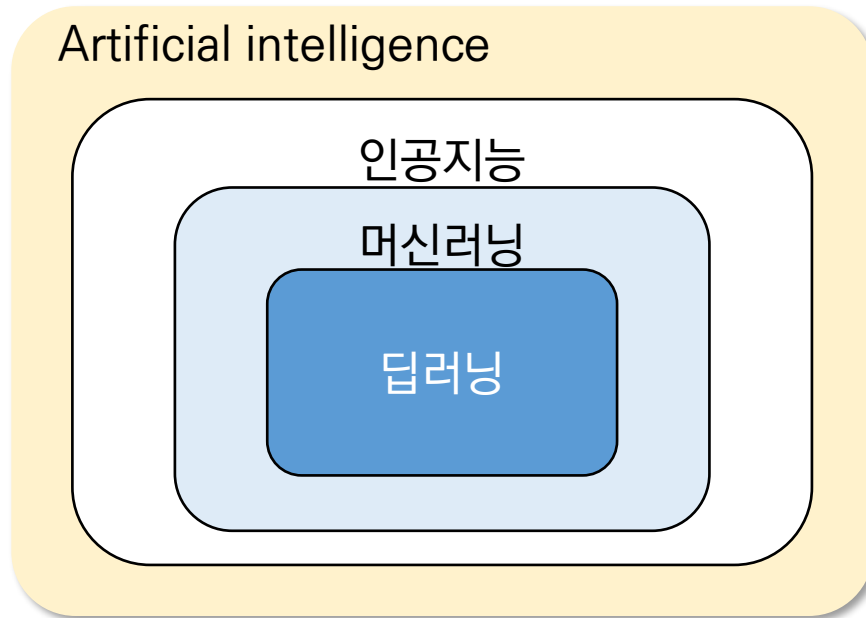
한국화학연구원
디지털화학연구센터
장 승 훈

교육 순서

- 기초 수학 (14:00 -)
 - 선형 대수
- 지도학습 (15:00 -)
 - Classification
 - Regression
- 실습 (16:00 -)
 - 실습

기초 수학 - 선형대수

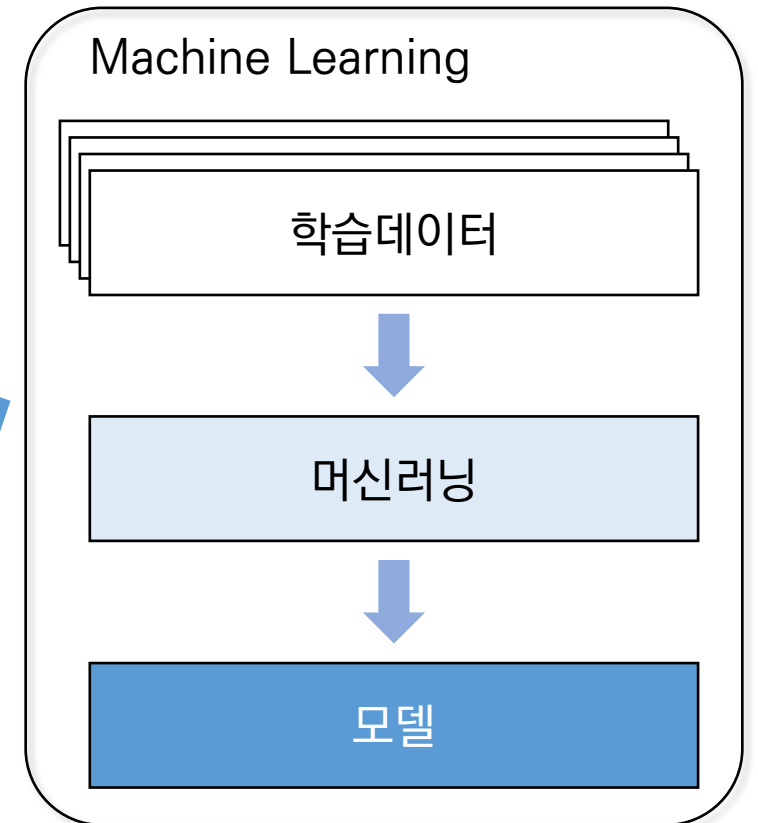
기계학습 (Machine Learning) 개요



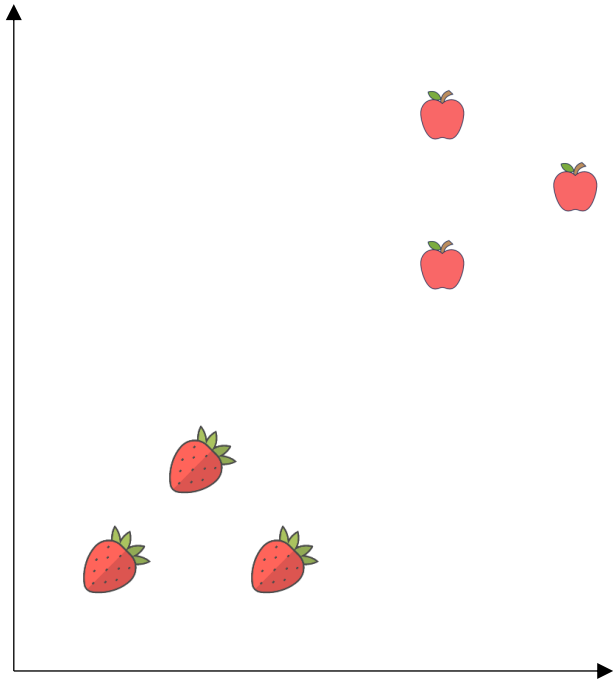
인공지능 : 인간의 지능을 갖고 있는 지능을 갖춘 컴퓨터 시스템, 인간의 지능을 기계 등에 인공적으로 구현한 것. **지식기반 프로그래밍** 또는 **규칙 기반 인공지능**, 반복적인 업무를 수행하는 사무자동화에 많이 사용.

머신러닝 : 컴퓨터가 학습모형을 기반으로 외부에서 주어진 **데이터를 통해 패턴을 파악하여 학습**. **인공지능의 한 분야**.

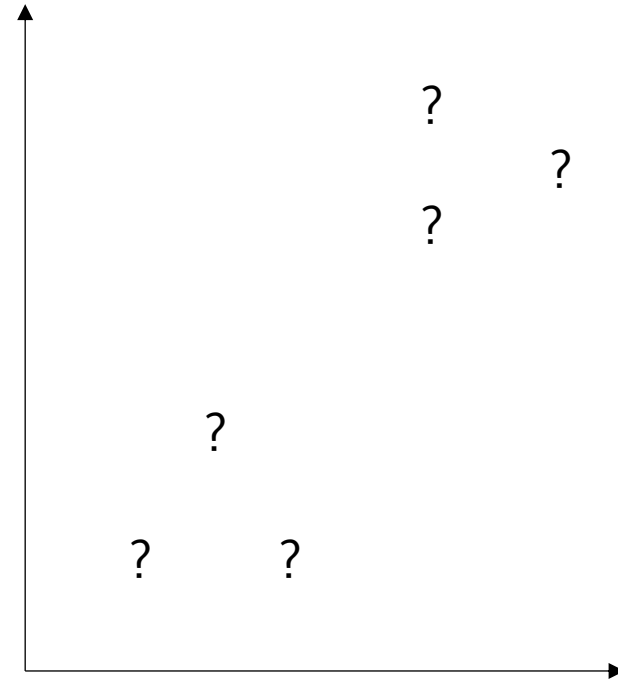
딥러닝 : 머신러닝에서 한 단계 더 발전한 형태, **머신러닝의 한 분야**, **인공신경망 구조를 기반으로 학습하는 방법**. (입력층 → 은닉층(2개 이상) → 출력층)



지도학습 vs 비지도학습



지도학습
Supervised Learning



비지도학습
Unsupervised Learning

지도학습 vs 비지도학습

맛	모양	과일이름
3	1	딸기
5	4	사과
1	1	딸기
2	2	딸기
4	6	사과
5	5	사과

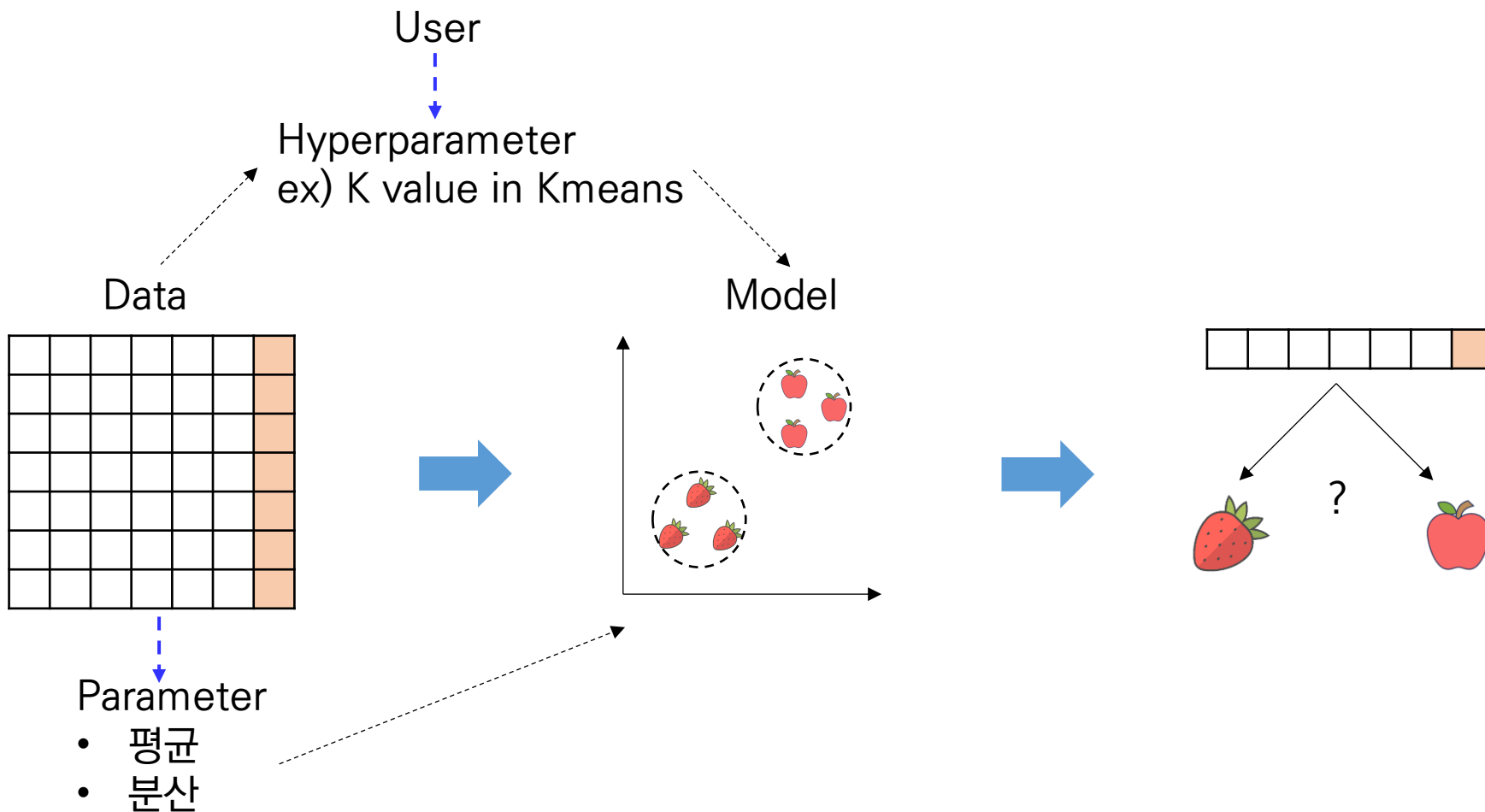
지도학습
Supervised Learning

맛	모양
3	1
5	4
1	1
2	2
4	6
5	5

비지도학습
Unsupervised Learning

- Dataset
- # of data
- Target data
- Feature data
- Classification (과일이름)
- Regression (가격예측)

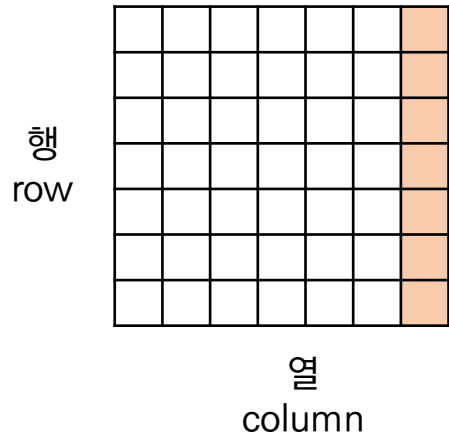
기계학습의 전반적인 과정



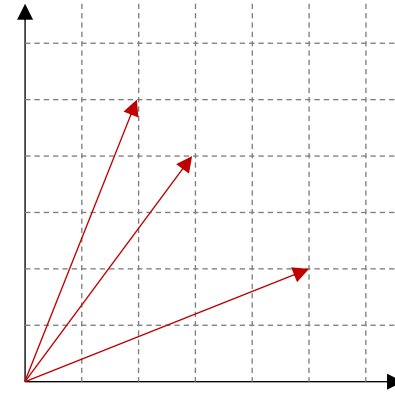
선형대수 & 머신러닝

- 데이터는 머신러닝의 필수 요소.
- 데이터를 잘 다룰 수 있다면 머신러닝을 수행하기 유리.
- 데이터를 잘 다루기 위해서 행렬에 대한 이해가 필요.
- 이는 곧 선형대수와 연관.
- 행렬 연산과 같은 선형대수의 여러 기본 개념 공부.

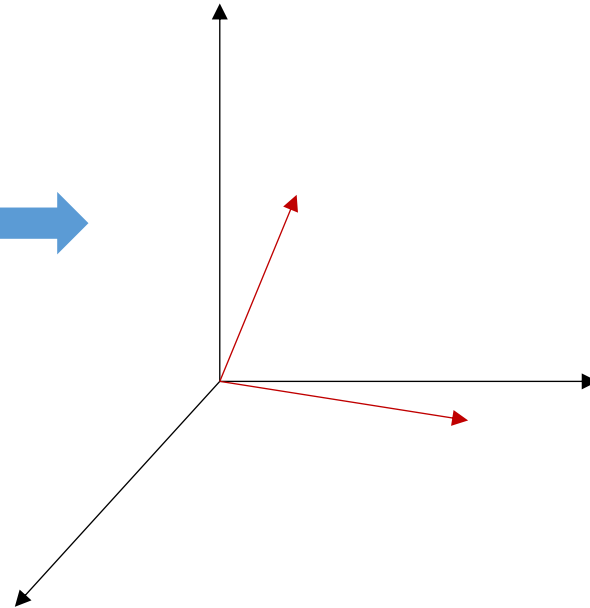
기초수학-선형대수-행렬



$$A = \begin{pmatrix} 1 & 5 \\ 3 & 4 \\ 5 & 2 \end{pmatrix}$$



$$A = \begin{pmatrix} 1 & 5 \\ 3 & 4 \\ 5 & 2 \end{pmatrix}$$



기초수학-선형대수-행렬

$$A = \begin{pmatrix} 1 & 5 \\ 3 & 4 \\ \textcolor{red}{5} & 2 \end{pmatrix} \quad A \text{행렬의 크기는 } 3 \times 2$$

$$a_{ij} \quad \begin{array}{l} \text{행렬원소 표시} \\ i: \text{행번호} \\ j: \text{열번호} \end{array} \quad \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ \textcolor{red}{a_{31}} & a_{32} \end{pmatrix}$$

기초수학-선형대수-행렬

스칼라
Scalar

1

벡터
Vector

$\begin{pmatrix} 1 \\ 3 \end{pmatrix}$

행렬
Matrix

$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$

텐서
Tensor

$\begin{pmatrix} \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} & \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \\ \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} & \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \end{pmatrix}$

특별한 수학적인 그릇



숫자 데이터를 다양한 차원 구조로 담을 수 있는 수학적인 그릇

기초수학-선형대수-대각행렬

대각행렬
Diagonal matrix

$$D = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 7 \end{pmatrix}$$

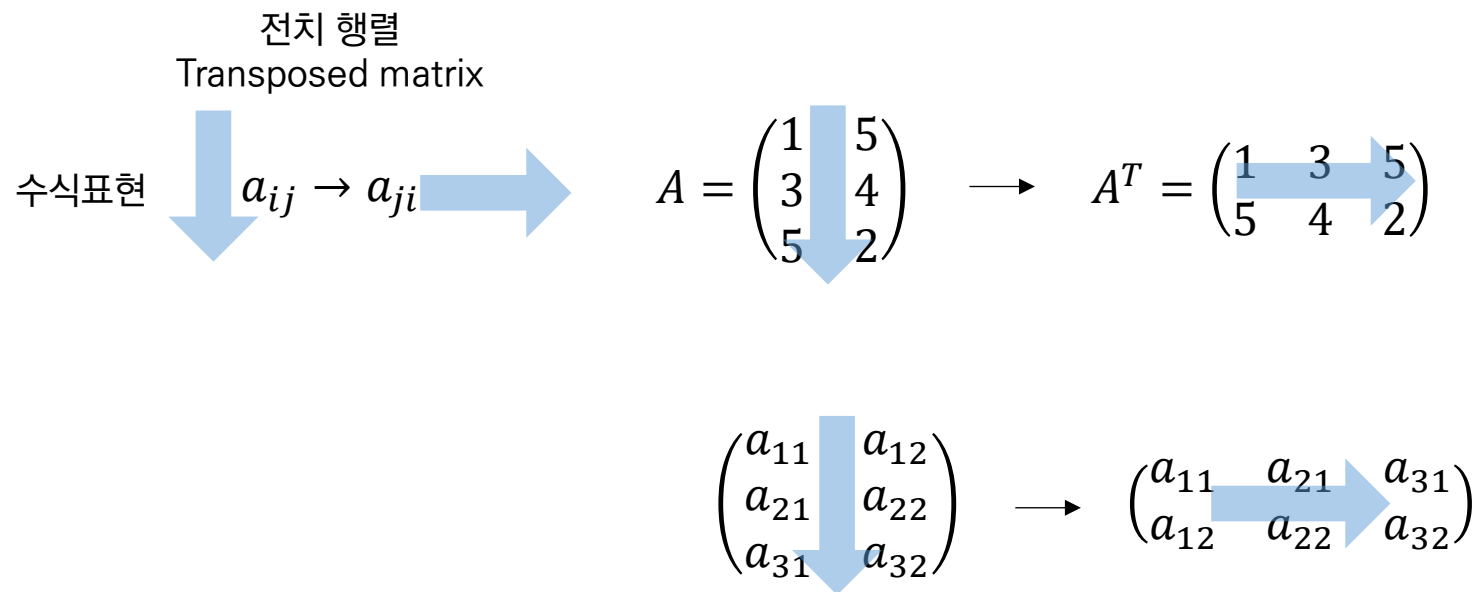
단위행렬
Identity matrix

$$I = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

정사각행렬
(square matrix)

$$D = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 7 \end{pmatrix}$$

기초수학-선형대수-전치행렬



기초수학-선형대수-행렬연산

행렬 덧셈, 뺄셈

$$\begin{pmatrix} 1 & 5 \\ 3 & 4 \\ 5 & 2 \end{pmatrix} + \begin{pmatrix} 0 & 1 \\ 2 & 1 \\ -1 & 0 \end{pmatrix} = \begin{pmatrix} 1+0 & 5+1 \\ 3+2 & 4+1 \\ 5-1 & 2+0 \end{pmatrix} = \begin{pmatrix} 1 & 6 \\ 5 & 5 \\ 4 & 2 \end{pmatrix}$$

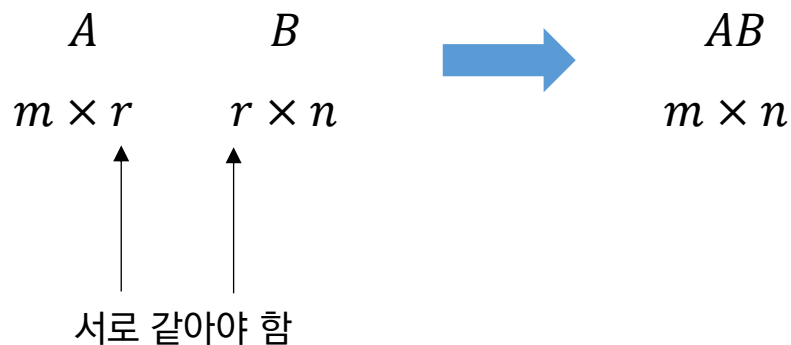
$$\begin{pmatrix} 1 & 5 \\ 3 & 4 \\ 5 & 2 \end{pmatrix} - \begin{pmatrix} 0 & 1 \\ 2 & 1 \\ -1 & 0 \end{pmatrix} = \begin{pmatrix} 1-0 & 5-1 \\ 3-2 & 4-1 \\ 5+1 & 2-0 \end{pmatrix} = \begin{pmatrix} 1 & 4 \\ 1 & 3 \\ 6 & 2 \end{pmatrix}$$

행렬 스칼라곱

$$2A = 2 \begin{pmatrix} 1 & 5 \\ 3 & 4 \\ 5 & 2 \end{pmatrix} = \begin{pmatrix} 2 \cdot 1 & 2 \cdot 5 \\ 2 \cdot 3 & 2 \cdot 4 \\ 2 \cdot 5 & 2 \cdot 2 \end{pmatrix} = \begin{pmatrix} 2 & 10 \\ 6 & 8 \\ 10 & 4 \end{pmatrix}$$

기초수학-선형대수-행렬연산

행렬 곱 (matrix multiplication)
-특정 조건을 만족해야 연산 가능.



$$AB = \begin{pmatrix} 2 & 5 \\ 3 & 4 \\ 5 & 2 \end{pmatrix} \begin{pmatrix} 3 & 0 & 2 \\ 1 & 3 & 1 \end{pmatrix}$$

$$= \begin{pmatrix} 2 \cdot 3 + 5 \cdot 1 & 2 \cdot 0 + 5 \cdot 3 & 2 \cdot 2 + 5 \cdot 1 \\ 3 \cdot 3 + 4 \cdot 1 & 3 \cdot 0 + 4 \cdot 3 & 3 \cdot 2 + 4 \cdot 1 \\ 5 \cdot 3 + 2 \cdot 1 & 5 \cdot 0 + 2 \cdot 3 & 5 \cdot 2 + 2 \cdot 1 \end{pmatrix}$$

$$= \begin{pmatrix} 11 & 15 & 9 \\ 13 & 12 & 10 \\ 17 & 6 & 12 \end{pmatrix}$$

기초수학-선형대수-행렬연산

행렬의 원소곱 (matrix element multiplication)

-차원이 동일한 두 행렬의 동일 위치 원소를 서로 곱함

$$(A \odot B)_{ij} = a_{ij}b_{ij}$$

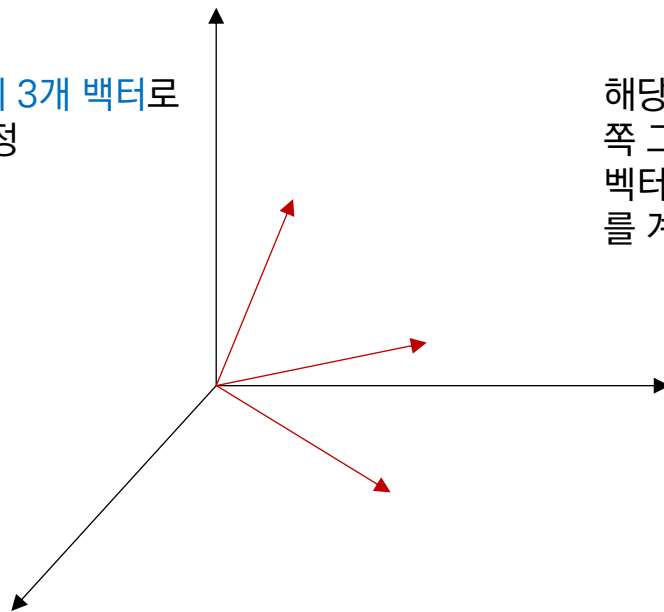
$$A \odot B = \begin{pmatrix} 1 & 5 \\ 3 & 4 \\ 5 & 2 \end{pmatrix} \odot \begin{pmatrix} 2 & 1 \\ 3 & 2 \\ 1 & 4 \end{pmatrix} = \begin{pmatrix} 1 \times 2 & 5 \times 1 \\ 3 \times 3 & 4 \times 2 \\ 5 \times 1 & 2 \times 4 \end{pmatrix} = \begin{pmatrix} 2 & 5 \\ 9 & 8 \\ 5 & 8 \end{pmatrix}$$

기초수학-선형대수-행렬식

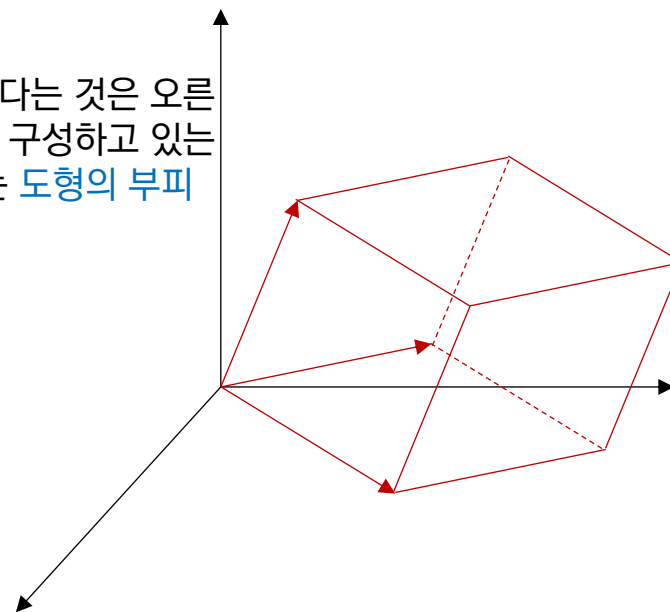
행렬식 (determinant)

- 행렬의 특성을 하나의 숫자로 표현하는 방법의 하나.

행렬이 3차원 공간의 3개 벡터로
구성되어 있다고 가정



해당 행렬식을 구한다는 것은 오른쪽
그림처럼 행렬을 구성하고 있는
벡터로 만들 수 있는 도형의 부피
를 계산한다는 의미



기초수학-선형대수-행렬식

1x1 행렬 A

$$\begin{bmatrix} a_{11} \end{bmatrix}$$

$$\det(A) = a_{11}$$

2x2 행렬 A

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$$

$$\det(A) = a_{11}a_{22} - a_{12}a_{21}$$

3x3 행렬 A

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

M_{ij} 는 i행과 j열을 제외한
소행렬(Minor Matrix)의 행렬식

$$\det(A) = a_{11}M_{11} - a_{12}M_{12} + a_{13}M_{13}$$

$$= a_{11} \begin{vmatrix} a_{22} & a_{23} \\ a_{32} & a_{33} \end{vmatrix} - a_{12} \begin{vmatrix} a_{21} & a_{23} \\ a_{31} & a_{33} \end{vmatrix} + a_{13} \begin{vmatrix} a_{21} & a_{22} \\ a_{31} & a_{32} \end{vmatrix}$$

$$= a_{11}(a_{22}a_{33} - a_{23}a_{32}) - a_{12}(a_{21}a_{33} - a_{23}a_{31}) + a_{13}(a_{21}a_{32} - a_{22}a_{31})$$

$$= a_{11}a_{22}a_{33} + a_{11}a_{23}a_{31} + a_{13}a_{21}a_{32} - a_{11}a_{23}a_{32} - a_{12}a_{21}a_{33} - a_{13}a_{22}a_{31}$$

기초수학-선형대수-역행렬

역행렬 (inverse matrix)

- 역수와 비슷한 개념으로 행렬 A 에 대해서 $AB=I$ 를 만족하는 행렬 B 가 존재한다는 의미.

$$AA^{-1} = A^{-1}A = I$$

- 역행렬을 구하려는 행렬의 행렬식이 0 이라면 역행렬은 존재하지 않음.
- 역행렬이 존재하는 행렬을 가역 행렬(Invertible matrix)이라고 부름.

기초수학-선형대수-역행렬

우리가 주로 보는 2×2 행렬에 대한 역행렬은

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}^{-1} = \frac{1}{A_{11}A_{22} - A_{12}A_{21}} \begin{bmatrix} A_{22} & -A_{12} \\ -A_{21} & A_{11} \end{bmatrix}$$

이고, 3×3 행렬에 대한 역행렬은

$$\begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix}^{-1} = \frac{1}{K} \begin{bmatrix} A_{22}A_{33} - A_{23}A_{32} & A_{13}A_{32} - A_{12}A_{33} & A_{12}A_{23} - A_{13}A_{22} \\ A_{21}A_{33} - A_{23}A_{31} & A_{11}A_{33} - A_{13}A_{31} & A_{13}A_{21} - A_{11}A_{23} \\ A_{21}A_{32} - A_{22}A_{31} & A_{12}A_{31} - A_{11}A_{32} & A_{11}A_{22} - A_{12}A_{21} \end{bmatrix}$$

이며, 여기서

$$K = A_{11}A_{22}A_{33} - A_{11}A_{23}A_{32} - A_{12}A_{21}A_{33} + A_{12}A_{23}A_{31} + A_{13}A_{21}A_{32} - A_{13}A_{22}A_{31}$$

기초수학-선형대수-내적

내적 (inner product)

- 다른 연산과 같은 하나의 연산
- 벡터와 벡터의 연산을 통해 스칼라 값을 얻게 됨. (보통, 스칼라를 더하면 스칼라, 벡터를 더하면 벡터가 됨.)

$$\langle \mathbf{n}, \mathbf{v} \rangle = \mathbf{n} \cdot \mathbf{v} = n_1 v_1 + u_2 v_2 + \cdots + u_n v_n$$

- 내적을 구하려는 각 벡터의 요소를 서로 곱한 후 더하는 것으로 내적 값을 구할 수 있음.
- 위 식을 벡터 행태로 표현하면 아래와 같음.

$$\mathbf{u} = \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix}, \mathbf{v} = \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix} \quad \mathbf{u} \cdot \mathbf{v} = \mathbf{u}^T \mathbf{v}$$

$$\begin{aligned} \mathbf{u} &= \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \mathbf{v} = \begin{pmatrix} 3 \\ 2 \\ 3 \end{pmatrix} & \mathbf{u} \cdot \mathbf{v} &= \mathbf{u}^T \mathbf{v} \\ & & \mathbf{u} \cdot \mathbf{v} &= 1 \times 3 + 2 \times 2 + 3 \times 3 \\ & & &= 3 + 4 + 9 \\ & & &= 16 \end{aligned}$$

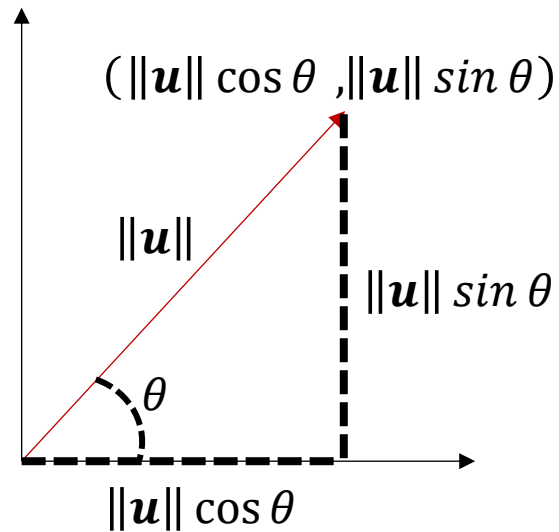
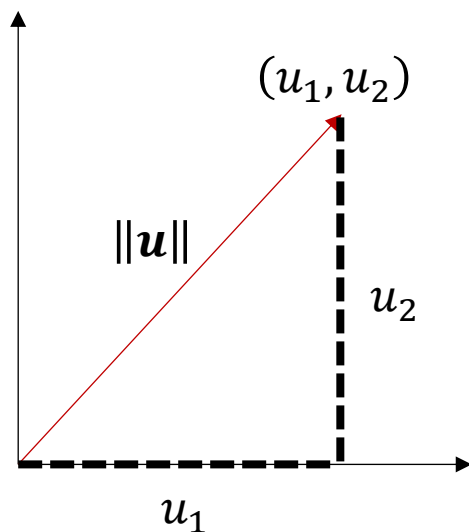
기초수학-선형대수-내적 활용

두 벡터의 각도

- (1) 내적 > 0 이면, 두 벡터 사이의 각도 < 90
- (2) 내적 < 0 이면, 두 벡터 사이의 각도 > 90
- (3) 내적 $= 0$ 이면, 두 벡터 사이의 각도 $= 90$

벡터의 길이

$$\mathbf{u} = (u_1, u_2, u_3, \dots, u_n) \quad \|\mathbf{u}\| = \sqrt{u_1^2 + u_2^2 + \dots + u_n^2}$$

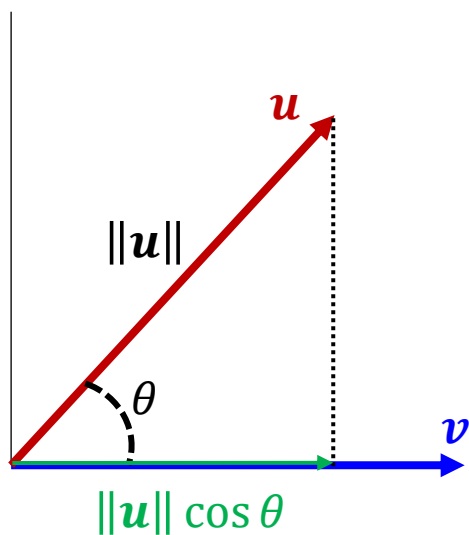


기초수학-선형대수-내적 활용

두 벡터의 길이와 각도 θ 를 안다면 내적 값을 구할 수 있음.

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta$$

$$\cos \theta = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

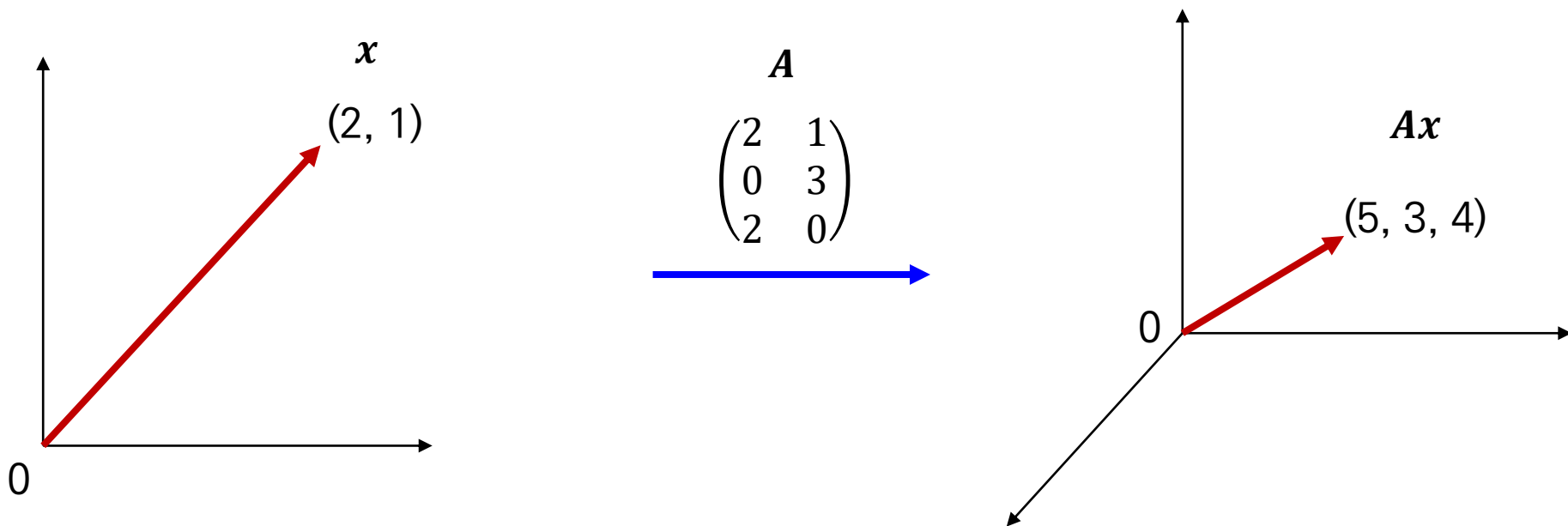


두 벡터의 내적은 벡터 $\mathbf{u}$ 를 벡터 $\mathbf{v}$ 에 정사영 시킨 벡터의 길이, 즉 $\|\mathbf{u}\| \cos \theta$ 과 기존벡터 $\mathbf{v}$ 의 길이인 $\|\mathbf{v}\|$ 의 곱과 같음.

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta$$

기초수학-선형대수-선형 변환

선형 변환 (linear transformation)은 두 벡터 공간 사이의 함수
2차원 공간의 $\begin{pmatrix} 2 \\ 1 \end{pmatrix}$ 벡터에 선형변환 A 를 적용한 결과 3차원 벡터 $\begin{pmatrix} 5 \\ 3 \\ 4 \end{pmatrix}$ 가 되었음. 아래 그림 참고
우리가 흔히 사용하던 행렬은 **선형 변환의 의미**를 가지고 있음.



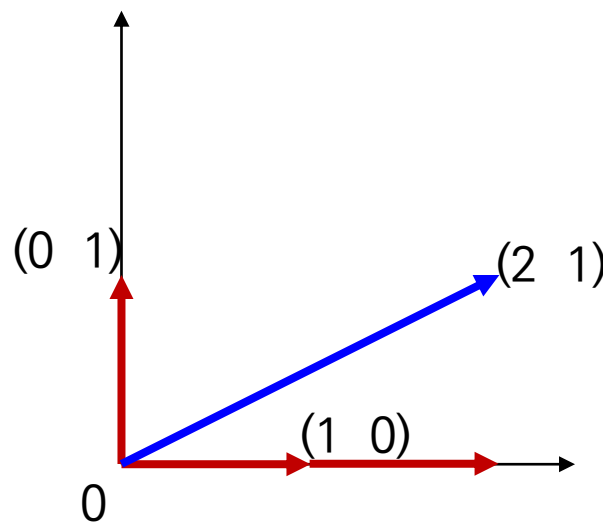
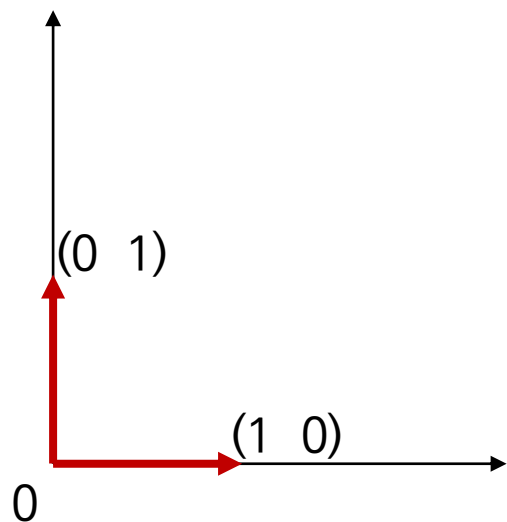
$$Ax = \begin{pmatrix} 2 & 1 \\ 0 & 3 \\ 2 & 0 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 5 \\ 3 \\ 4 \end{pmatrix}$$

기초수학-선형대수-벡터공간, 기저

$(1\ 0)$, $(0\ 1)$ 두 벡터는 2차원 공간의 기저가 될 수 있음.

즉, 2차원 공간에 존재하는 모든 점은 두 기저 벡터 $(1\ 0)$, $(0\ 1)$ 로 표현할 수 있음.

2개의 기저 벡터로 span 할 수 있는 벡터 공간은 2차원



$$(2\ 1) = 2(1\ 0) + 1(0\ 1)$$

기초수학-선형대수-벡터공간, 기저

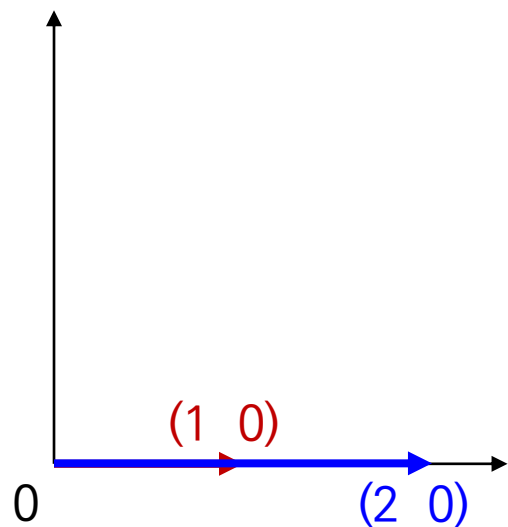
$(1\ 0)$, $(2\ 0)$ 두 벡터로는 2차원 평면의 기저가 될 수 없음.

벡터는 2개이지만 서로 **선형 독립이 아님**.

선형 독립(linear independent)이란 하나의 벡터로 다른 벡터를 표현할 수 없다는 뜻. $(1\ 0)$ $(0\ 1)$

선형 독립이 아니란 말은 벡터 사이에 선형관계가 존재하며,

서로 의존적이라 자신을 다른 벡터를 이용해 표현할 수 있다는 뜻

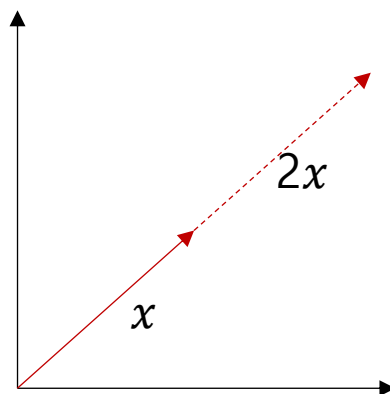
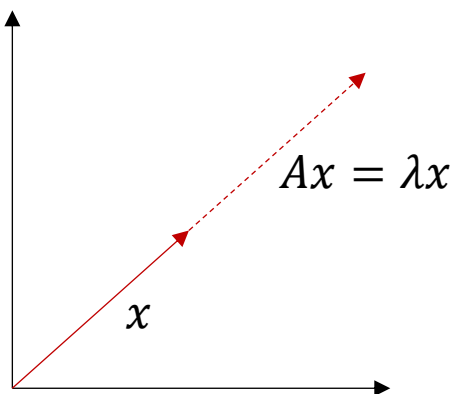


→ 벡터가 2개더라도 span 할 수 있는 공간은 1차원 직선에 불과함.

기초수학-선형대수-고유값, 고유벡터

$$Ax = \lambda x$$

- 벡터 x 에 선형 변환 A 를 취했는데, 그 결과는 기존 벡터의 방향은 변하지 않고 길이가 λ 만큼 변했다는 것을 의미
- 고유벡터 (eigenvector)란 벡터에 선형 변환을 취했을 때, 방향은 변하지 않고 크기만 변하는 벡터를 의미
- 선형 변환 이후 변한 크기가 고유값 (eigenvalue)을 의미.



기초수학-선형대수-이차식 표현

- 이차식 표현(quadratic form)은 다항식을 벡터 형태로 나타낼 때 사용하는 유용한 방법
- 머신러닝에서 수많은 벡터와 행렬을 다룰 때, 이차식 표현은 유용하게 사용됨.
- 먼저 다항식 표현에 대해서,

$$w_1x_1 + w_2x_2 + \cdots + w_px_p$$
$$(w_1, w_2, \cdots, w_p) \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_p \end{pmatrix} = w^T x$$

- 최고차항이 이차항일 때는?

$$w_1x_1^2 + w_2x_2^2 + \cdots + w_px_p^2$$

기초수학-선형대수-이차식 표현

- 최고차항이 이차항일 때는?

$$w_1 x_1^2 + w_2 x_2^2 + 2w_3 x_1 x_2$$

Cross product term

$$(x_1 \ x_2) \begin{pmatrix} w_1 & w_3 \\ w_3 & w_2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \mathbf{x}^T \mathbf{w} \mathbf{x}$$

- 벡터의 원소가 3개인 경우.

$$w_1 x_1^2 + w_2 x_2^2 + w_3 x_3^2 + 2w_4 x_1 x_2 + 2w_5 x_1 x_3 + 2w_6 x_2 x_3$$

Cross product term

$$(x_1 \ x_2 \ x_3) \begin{pmatrix} w_1 & w_4 & w_5 \\ w_4 & w_2 & w_6 \\ w_5 & w_6 & w_3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \mathbf{x}^T \mathbf{w} \mathbf{x}$$

- $\mathbf{w}$ 는 대칭행렬
- Cross product term 이 모두 0이라면?

기초수학-선형대수-벡터 미분

- 타깃 데이터 y 를 가중치 벡터 $\mathbf{x} = (x_1, x_2, \dots, x_p)^T$ 로 미분한다면 아래와 같이 표현함

$$\frac{\partial y}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial y}{\partial x_1} \\ \frac{\partial y}{\partial x_2} \\ \vdots \\ \frac{\partial y}{\partial x_p} \end{pmatrix}$$

- 머신러닝에서 우리의 관심 대상은 피쳐 데이터 벡터 $\mathbf{x}$ 에 가중치를 곱한 $\mathbf{w}^T \mathbf{x}$ 와 $\mathbf{x}^T \mathbf{A} \mathbf{x}$ 입니다.
- 이때 $\mathbf{A}$ 는 대칭 행렬(symmetric matrix) 입니다. 만약 타깃 $y = \mathbf{w}^T \mathbf{x} = \mathbf{x}^T \mathbf{w}$ 를 데이터 벡터 $\mathbf{x}$ 에 대해 미분한다고 하면 아래와 같은 결과가 나옴.

$$\frac{\partial y}{\partial \mathbf{x}} = \frac{\partial(\mathbf{w}^T \mathbf{x})}{\partial \mathbf{x}} = \frac{\partial(\mathbf{x}^T \mathbf{w})}{\partial \mathbf{x}} = \mathbf{w}$$

기초수학-선형대수-벡터 미분

- 좀 더 풀어서 쓰면,

$$\frac{\partial y}{\partial x_i} = \frac{\partial (w_1 x_1 + w_2 x_2 + \cdots + w_p x_p)}{\partial x_i} = w_i$$

- 정리하면,

$$\frac{\partial y}{\partial \mathbf{x}} = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_p \end{pmatrix} = \mathbf{w}$$

- 회귀분석에서는 아래와 같은 대칭행렬을 사용

$$\frac{\partial (\mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w})}{\partial \mathbf{w}} = 2 \mathbf{X}^T \mathbf{X} \mathbf{w}$$

- 만약 대칭행렬(symmetric matrix) A 에 대해서 $\mathbf{x}^T A \mathbf{x}$ 를 미분한다고 가정하면,

$$\frac{\partial (\mathbf{x}^T A \mathbf{x})}{\partial \mathbf{x}} = 2 A \mathbf{x}$$

벡터의 원소가 3개인 경우.

$$w_1 x_1^2 + w_2 x_2^2 + w_3 x_3^2 + 2w_4 x_1 x_2 + 2w_5 x_1 x_3 + 2w_6 x_2 x_3$$

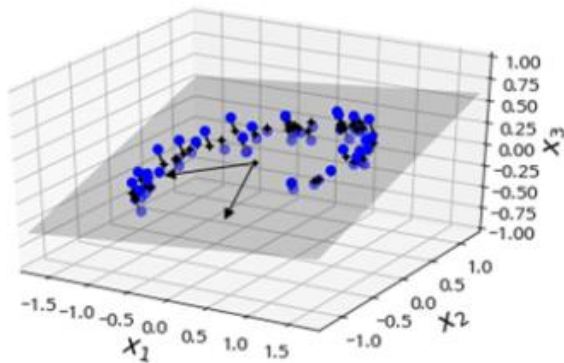
Cross product term

$$(x_1 \ x_2 \ x_3) \begin{pmatrix} w_1 & w_4 & w_5 \\ w_4 & w_2 & w_6 \\ w_5 & w_6 & w_3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \mathbf{x}^T \mathbf{w} \mathbf{x}$$

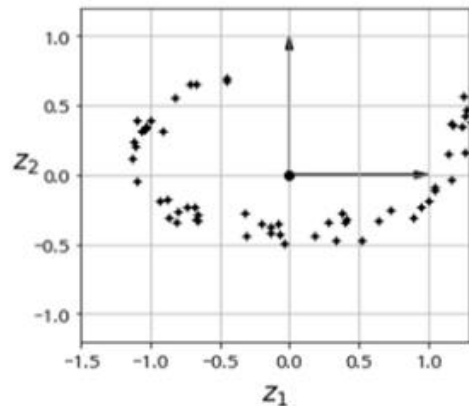
주성분 추출 → d차원으로 투영

$$X_{d\text{-proj}} = XW_d$$

- $X_{d\text{-proj}}$ (60,2) - d 차원으로 축소된 데이터셋
- X (60,3) - 원래 데이터셋
- W_d (3,2) - V 의 첫 d열로 구성된 행렬



projection



분산을 최대한 유지하는 초평면 위에
원래 데이터셋을 투영 시킨 결과

결과를 보면 2D 투영결과는
원본 3D 데이터셋과 비슷해보임.

```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components = 2)
X2D = pca.fit_transform(X)
```



PCA 학습이 끝나면 component_ 속성에 주성분이 담겨 있음.

→ 첫번째 주성분의 단위벡터: `pca.components_.T[:,0]`

Locally Linear Embedding (LLE)

Equation 8-4. LLE step 1: linearly modeling local relationships

$$\widehat{\mathbf{W}} = \underset{\mathbf{W}}{\operatorname{argmin}} \sum_{i=1}^m \left(\mathbf{x}^{(i)} - \sum_{j=1}^m w_{i,j} \mathbf{x}^{(j)} \right)^2 \quad \mathbf{n_neighbors} = j$$

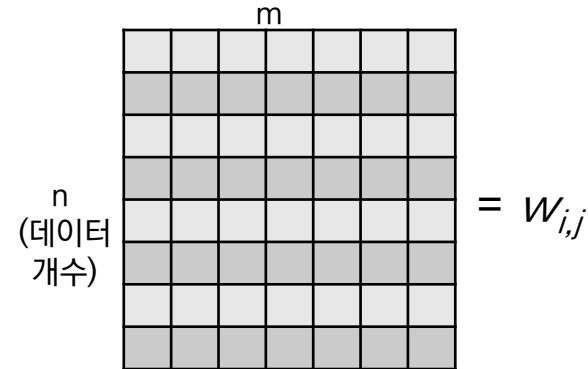
subject to $\begin{cases} w_{i,j} = 0 & \text{if } \mathbf{x}^{(j)} \text{ is not one of the } k \text{ c.n. of } \mathbf{x}^{(i)} \\ \sum_{j=1}^m w_{i,j} = 1 & \text{for } i = 1, 2, \dots, m \end{cases}$

가중치 정규화 (합은 1)

최적화된 가중치 전이

Equation 8-5. LLE step 2: reducing dimensionality while preserving relationships

$$\hat{\mathbf{Z}} = \underset{\mathbf{Z}}{\operatorname{argmin}} \sum_{i=1}^m \left(\mathbf{z}^{(i)} - \sum_{j=1}^m \hat{\mathbf{w}}_{i,j} \mathbf{z}^{(j)} \right)^2$$



훈련샘플 $x^{(i)}$ 에 대해 가장 가까운 m 개의 샘플을 찾는다.

선형함수로 $x^{(i)}$ 를 재구성 $\rightarrow \sum_{j=1}^m w_{i,j} x^{(j)}$

$x^{(i)}$ 와 $\sum_{j=1}^m w_{i,j} x^{(j)}$ 사이의 제곱거리가 최소로 되는 $w_{i,j}$ 를 찾는다.

$x^{(i)}$ 가 $x^{(j)}$ 의 가장 가까운 이웃 중 하나가 아니면 $w_{i,j}=0$.

$w_{i,j}$ 을 모두 담은 가중치 행렬은 은 훈련 샘플 사이에 있는 지역 선형 관계를 담고 있음

이러한 관계가 잘 보존되도록 훈련샘플을 저 차원 공간으로 mapping수행.

$z^{(j)}$ 는 d차원 공간에서 $x^{(j)}$ 의 상(image)이라고 할때

$$z^{(i)} \text{와 } \sum_{j=1}^m \hat{w}_{i,j} z^{(j)} \text{ 사이의 거리가 최소화 되어야 함.}$$

첫번째 단계와 비슷해 보이지만, 2번째 단계에서는 **가중치는 고정하고 저차원 공간에서 샘플 이미지의 최적 위치를 바꾸며** 찾아감.

Classification

개념, 성능 측정 지표

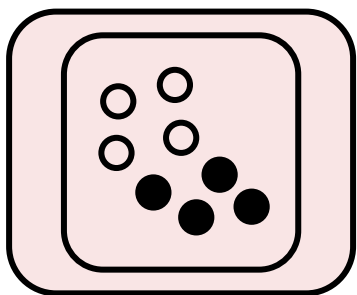
머신러닝 분류

머신러닝 시스템을

“학습하는 동안의 정보량(또는 감독의 유무)”에 따라 분류할 수 있음

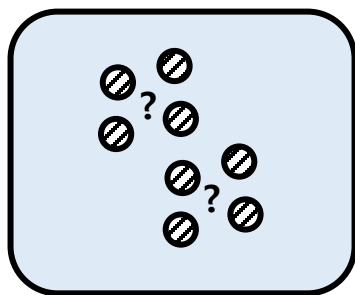
지도학습

분류
회귀
인공신경망



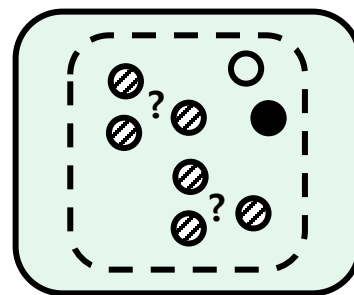
비지도학습

군집
시각화
차원축소



준지도학습

DBM
RBM



강화학습

Reinforcement
learning

대표적 Classification

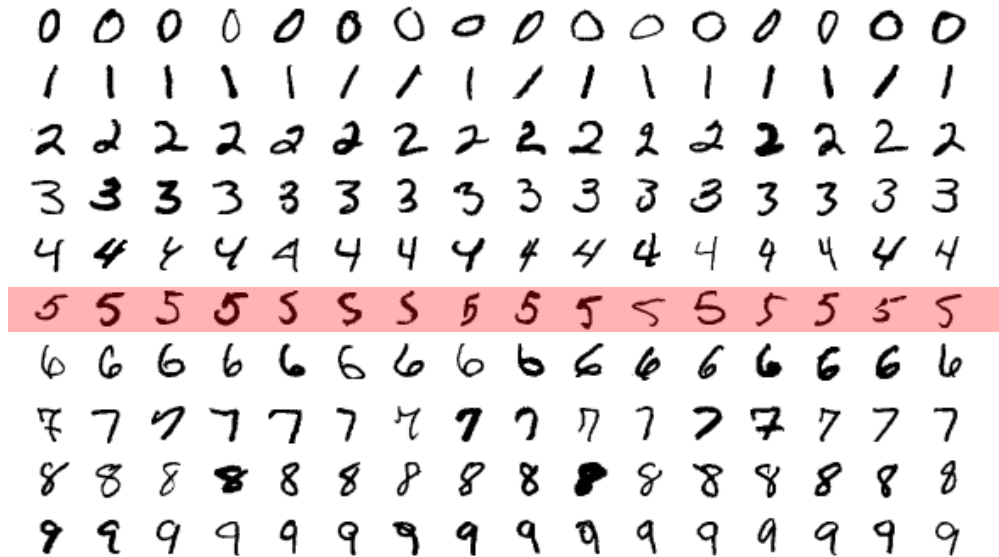
그림 1-5 지도 학습에서 레이블된 훈련 세트(예를 들면 스팸 분류)



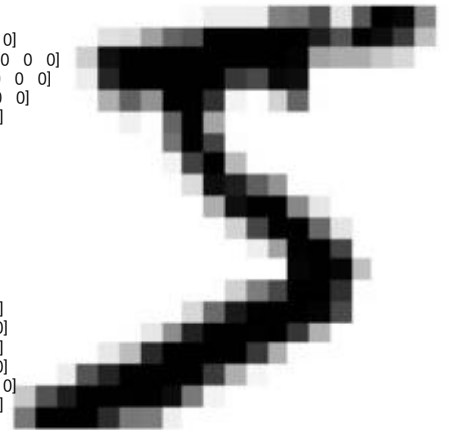
* Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow

MNIST Dataset을 활용한 분류 모델 실습

- 미국 고등학생들과 인구조사국 직원들이 **손으로** 쓴 작은 숫자 이미지.



위 그림과 같이 0에서 9까지 10가지로 분류될 수 있는 손글씨 숫자 이미지 70,000개로 구성.

[illegible]

각 이미지는 28×28 픽셀로 구성.
각 픽셀은 위와 같이 0~255 사이의 숫자 행렬로 표현.

MNIST Dataset

```
dict_keys(['data', 'target', 'frame', 'categories', 'feature_names', 'target_names', 'DESCR', 'details', 'url'])
```

```
{'data': array([[0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.],
                ...,
                [0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.])),
 'target': array(['5', '0', '4', ..., '4', '5', '6'], dtype=object),
 'frame': None,
 'categories': {},
 'feature_names': ['pixel1', 'pixel2', 'pixel3', 'pixel4', ..., 'pixel784'],
 'target_names': ['class'],
 'DESCR': "***Author**": Yann LeCun,,, **Source**": [MNIST Website](http://yan~
 'details': {'id': '554', 'name': 'mnist_784', 'version': '1',,,,}
 'url': 'https://www.openml.org/d/554'}
```



5 0 4 1 9 2 1 3 1 4
3 5 3 6 1 7 2 8 6 9
4 0 9 1 1 2 4 3 2 7
3 8 6 9 0 5 6 0 7 6
1 8 7 9 3 9 8 5 9 3
3 0 7 4 9 8 0 9 4 1
4 4 6 0 4 5 6 1 0 0
1 7 1 6 3 0 2 1 1 7
8 0 2 6 7 8 3 9 0 4
6 7 4 6 8 0 7 8 3 1

Binary Classification

- 금속/비금속
- 5/5아님
- 정상메일/스팸메일
- 코로나양성/음성

Binary Classification

- 금속/비금속
- 5/5아님
- 정상메일/스팸메일
- 코로나양성/음성



Confusion matrix

53057	1522
1325	4096

Confusion matrix

		예측	
		음성	양성
실제	음성	True Negative 	False Positive 
	양성	False Negative 	True Positive 

```
>>>from sklearn.metrics import confusion_matrix  
>>>confusion_matrix(y_train_5, y_train_pred)
```

정밀도(precision)

		예측	
		음성	양성
실제	음성	True Negative 8 7 2 9 3	False Positive 6
	양성	False Negative 5 5	True Positive 5 5 5

3/4

실제 '5'를 '5'라고 예측된 개수

$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$

'5'라고 예측된 총 개수

```
>>>from sklearn.metrics import precision_score, recall_score  
>>>precision_score(y_train_5, y_train_pred)
```

재현율(recall)

재현율(recall) or 민감도(sensitivity) or 진짜 양성 비율 (TPR)

		예측	
		음성	양성
실제	음성	True Negative 8 7 2 3 9	False Positive 6
	양성	False Negative 5 5	True Positive 5 5 5

← 3/5

```
>>>from sklearn.metrics import precision_score, recall_score  
>>>recall_score(y_train_5, y_train_pred)
```

실제 '5'를 '5'라고 예측된 개수

$$\text{Recall} = \frac{\text{True Positive}}{\text{False Negative} + \text{True Positive}}$$

실제 '5'가 예측에 사용된 총 개수

F1 score

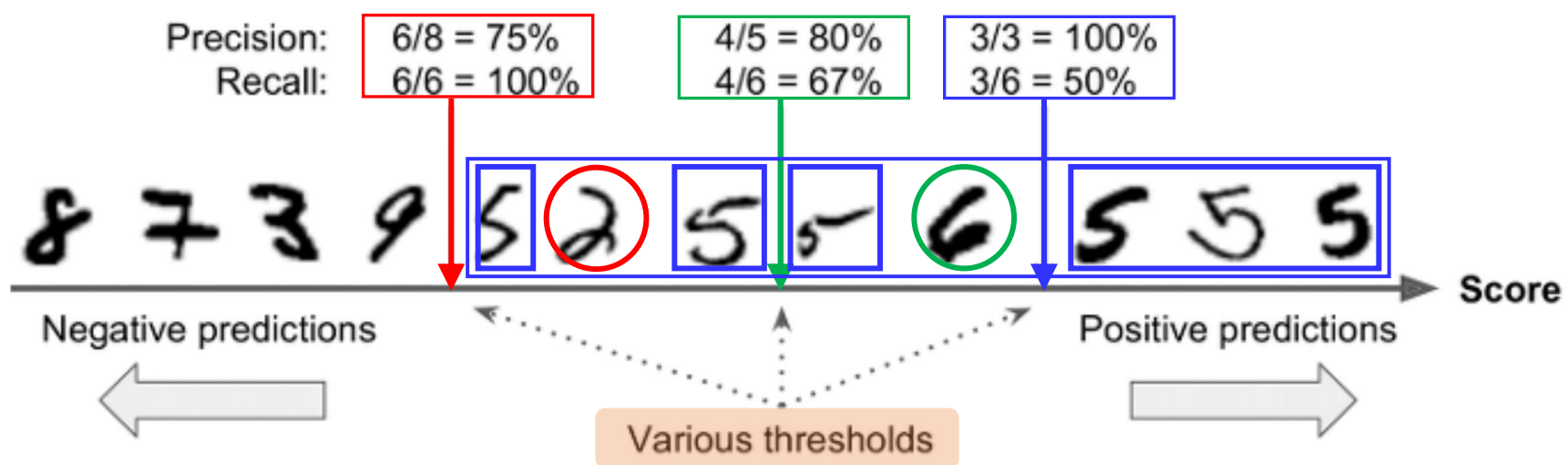
- 정밀도와 재현율을 하나의 숫자로 표현.
- 정밀도와 재현율의 조화평균 (harmonic mean)

$$\text{F1 score} = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}}$$

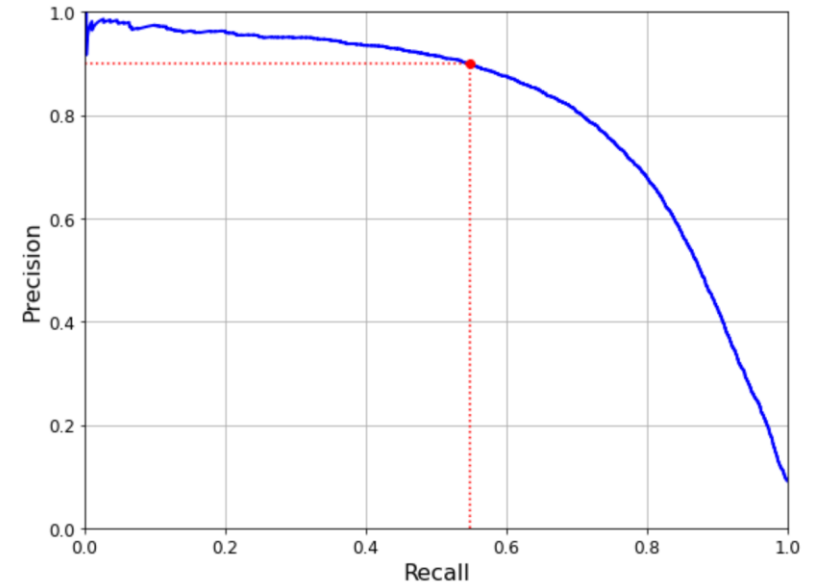
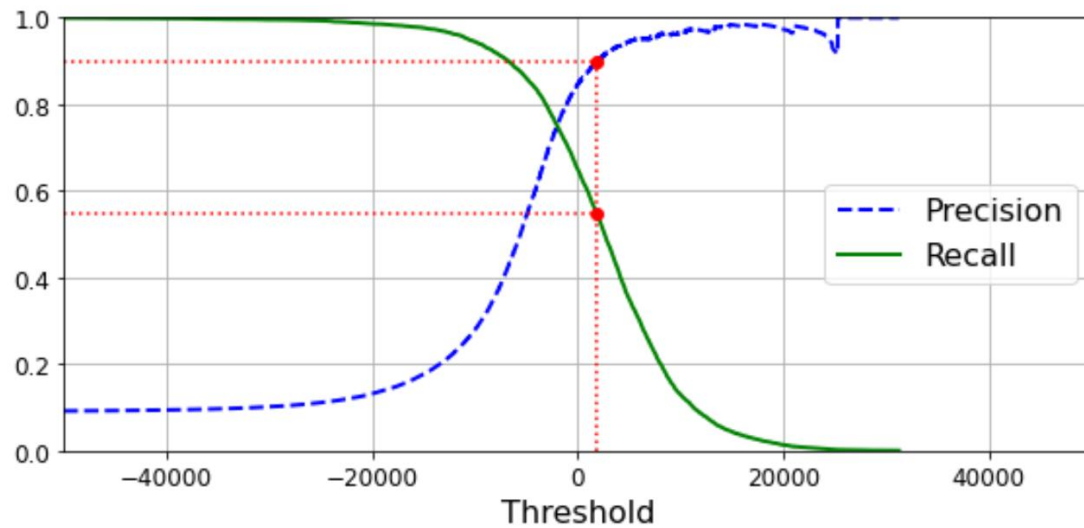
```
>>>from sklearn.metrics import f1_score  
>>>f1_score(y_train_5, y_train_pred)
```

Precision/Recall tradeoff

- 정밀도를 올리면 재현율이 줄어듦, 반대로 재현율을 올리면 정밀도가 줄어듦.
- 예시 : SGD Classifier
 - decision function을 통해 샘플의 점수를 계산
 - 점수가 **threshold** 보다 크면 양성클래스에 할당, 그렇지 않으면 음성 클래스로 할당



Precision/Recall tradeoff

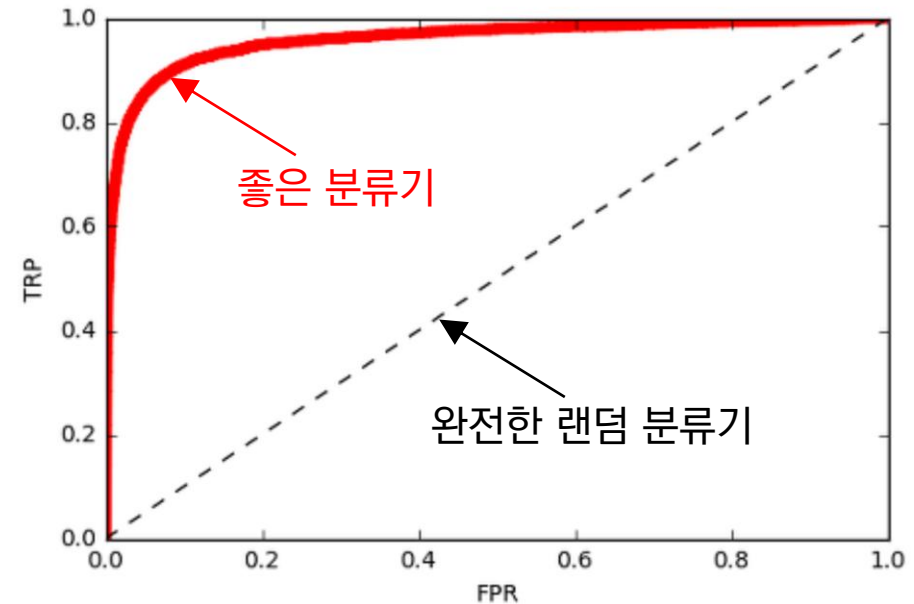


- 정밀도를 올리면 재현율이 줄어들고, 반대로 재현율을 올리면 정밀도가 줄어듦.

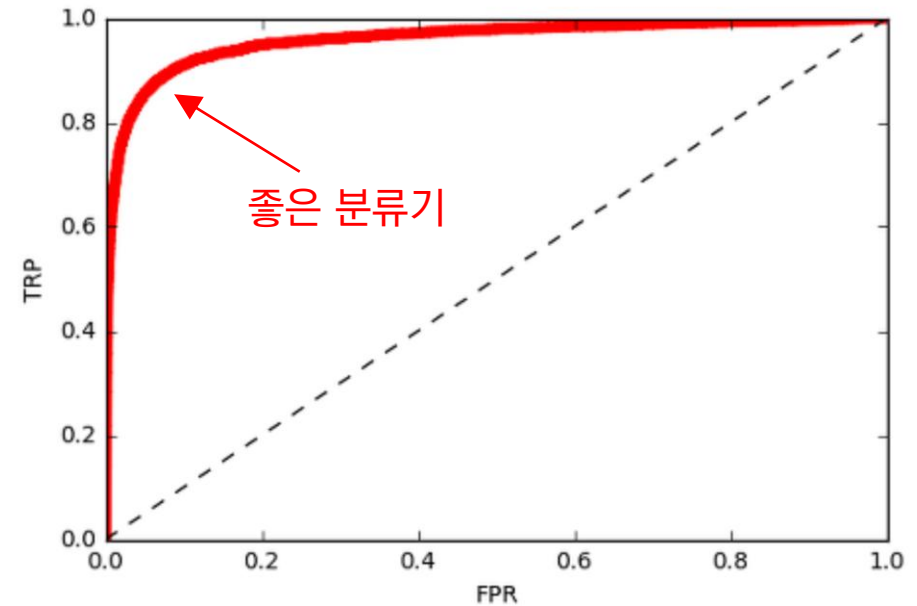
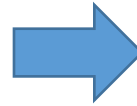
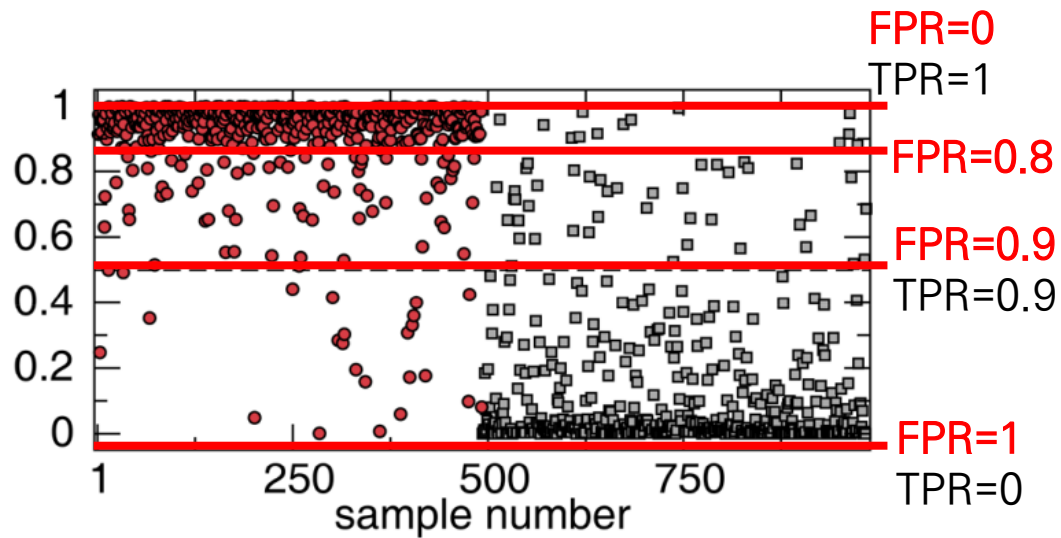
ROC (Receiver Operating Characteristic)

- 이진분류에서 널리 사용하는 도구.
- 거짓 양성 비율(FPR)에 대한 진짜 양성 비율(TPR, Recall)의 곡선.
- TPR이 높을수록 FPR이 늘어남.

		예측	
		음성	양성
실제	음성	True Negative 7 2 8 3 9	False Positive 6
	양성	False Negative 5 5	True Positive 5 5 5

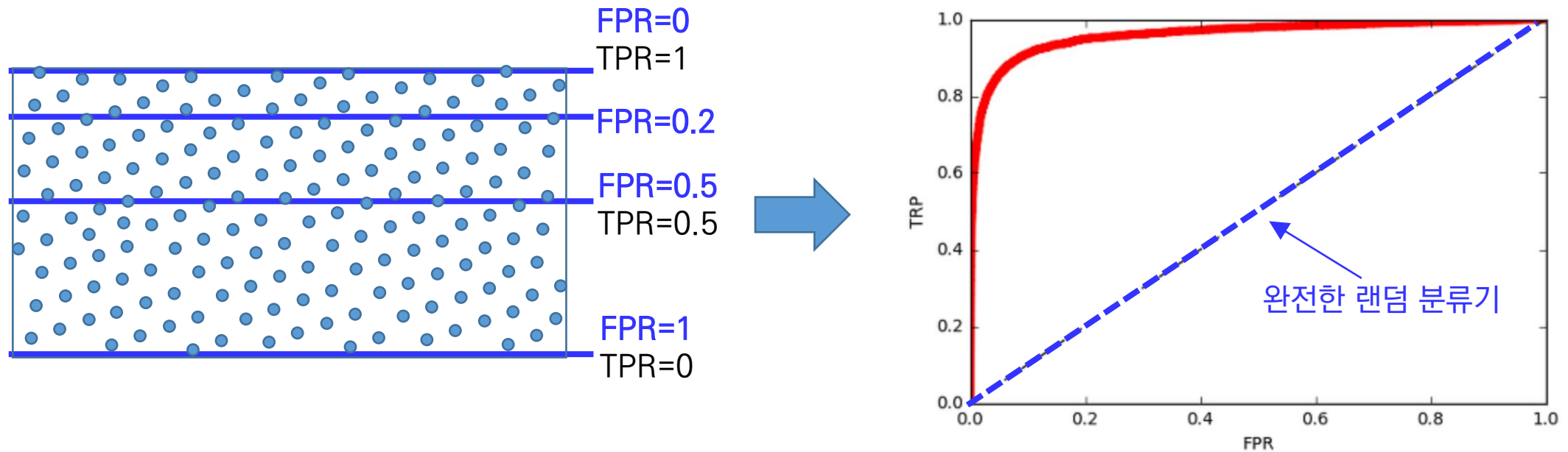


ROC (Receiver Operating Characteristic)



Threshold를 바꿔가며 FPR, TPR 체크

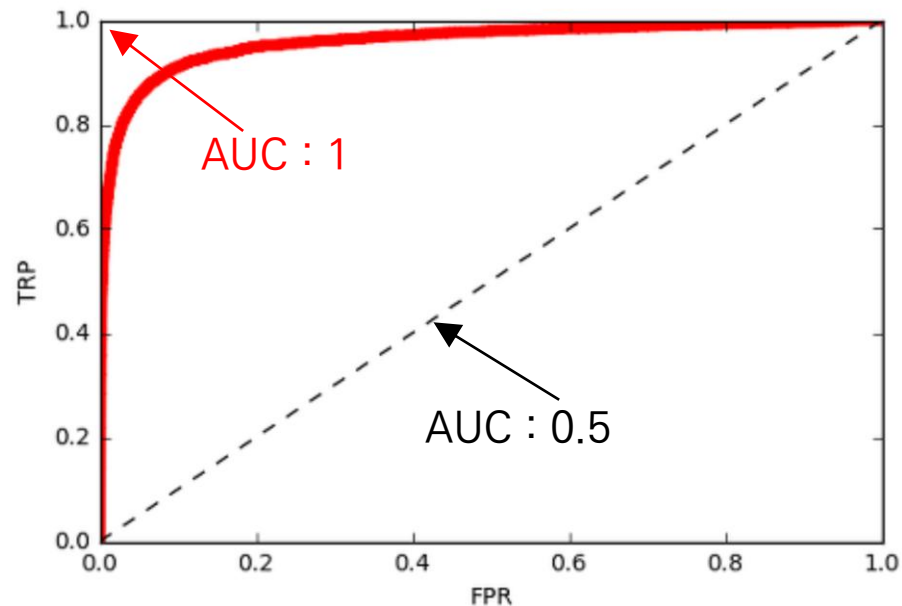
ROC (Receiver Operating Characteristic)



Threshold를 바꿔가며 FPR, TPR 체크

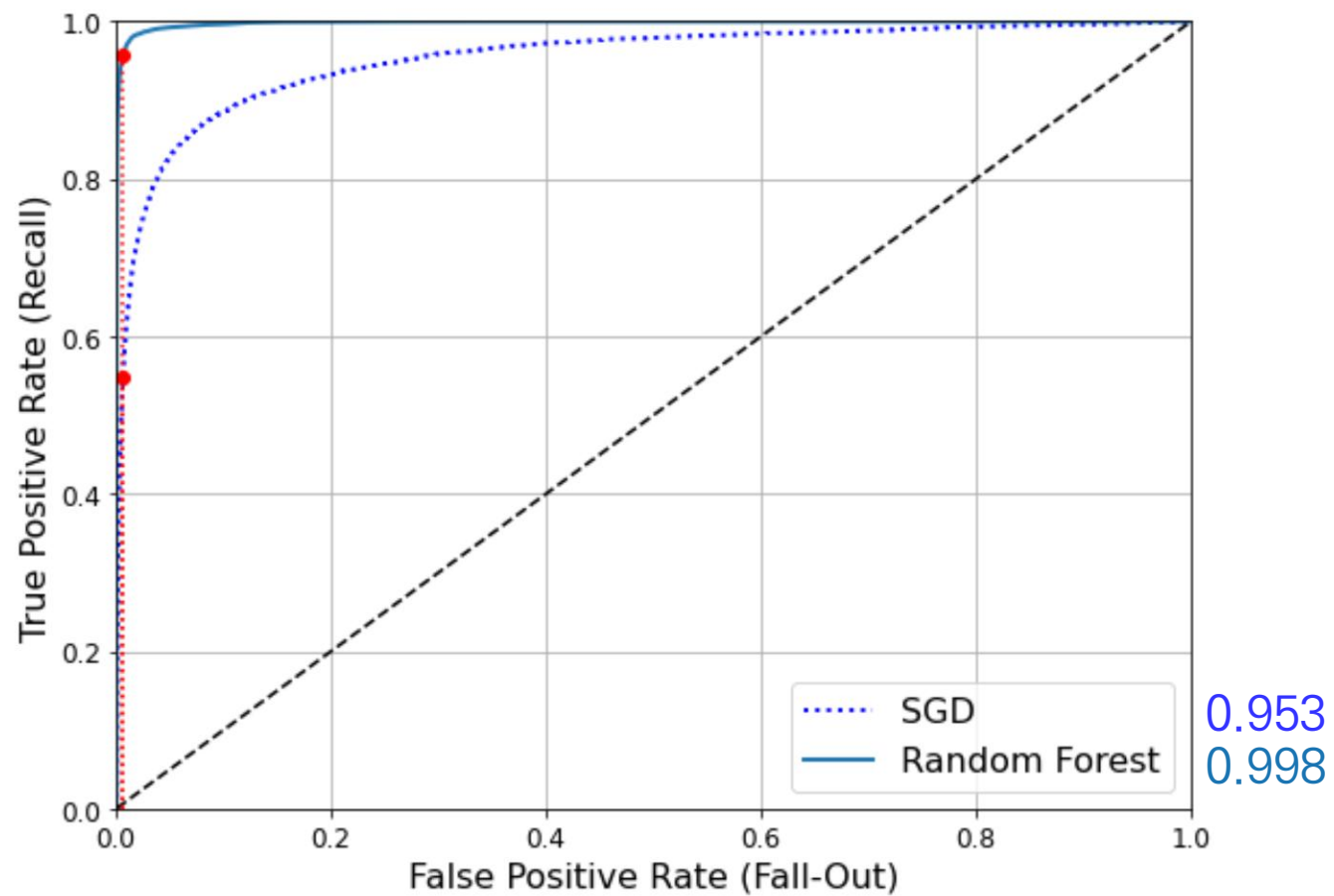
ROC AUC(Area Under the Curve) score

- ROC 곡선 아래 면적
- 완벽한 분류기의 ROC의 AUC는 1이며, 완전한 랜덤 분류기는 0.5.



```
>>>from sklearn.metrics import roc_auc_score  
>>>roc_auc_score(y_train_5, y_scores)
```

Classification 모델 성능 비교.



다중 분류

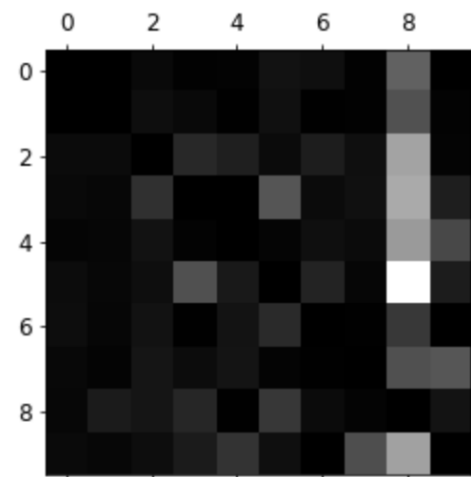
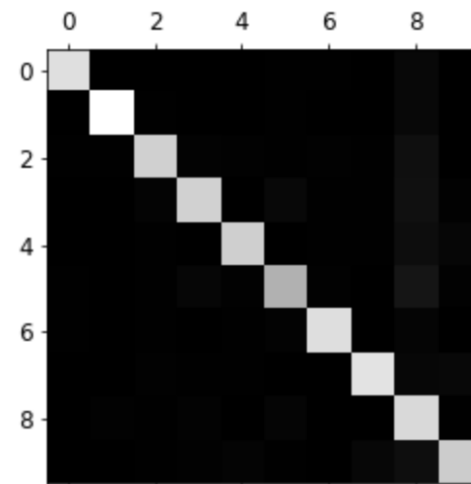
```
y_train_pred = cross_val_predict(sgd_clf, X_train_scaled, y_train, cv=3)
conf_mx = confusion_matrix(y_train, y_train_pred)
conf_mx
```

```
array([[5577,    0,   22,    5,    8,   43,   36,    6,  225,    1],
       [    0, 6400,   37,   24,    4,   44,    4,    7,  212,   10],
       [   27,   27, 5220,   92,   73,   27,   67,   36,  378,   11],
       [   22,   17,  117, 5227,    2,  203,   27,   40,  403,   73],
       [   12,   14,   41,    9, 5182,   12,   34,   27,  347,  164],
       [   27,   15,   30,  168,   53, 4444,   75,   14,  535,   60],
       [   30,   15,   42,    3,   44,   97, 5552,    3,  131,    1],
       [   21,   10,   51,   30,   49,   12,    3, 5684,  195,  210],
       [   17,   63,   48,   86,    3,  126,   25,   10, 5429,   44],
       [   25,   18,   30,   64,  118,   36,    1,  179,  371, 5107]])
```

```
y_train_pred = cross_val_predict(sgd_clf, X_train_scaled, y_train, cv=3)
conf_mx = confusion_matrix(y_train, y_train_pred)
conf_mx
```

```
array([[5577,    0,   22,    5,    8,   43,   36,    6,  225,    1],
       [    0, 6400,   37,   24,    4,   44,    4,    7,  212,   10],
       [   27,   27, 5220,   92,   73,   27,   67,   36,  378,   11],
       [   22,   17,  117, 5227,    2,  203,   27,   40,  403,   73],
       [   12,   14,   41,    9, 5182,   12,   34,   27,  347,  164],
       [   27,   15,   30,  168,   53, 4444,   75,   14,  535,   60],
       [   30,   15,   42,    3,   44,   97, 5552,    3,  131,    1],
       [   21,   10,   51,   30,   49,   12,    3, 5684,  195,  210],
       [   17,   63,   48,   86,    3,  126,   25,   10, 5429,   44],
       [   25,   18,   30,   64,  118,   36,    1,  179,  371, 5107]])
```

		예측	
		음성	양성
실제	음성	True Negative 7 2 8 3 9	False Positive 6
	양성	False Negative 5 5	True Positive 5 5 5



실제 8이 8로 적절히 분류되었음.

많은 이미지가 8로 잘못 분류되었음을 의미

연구에 활용된 classification

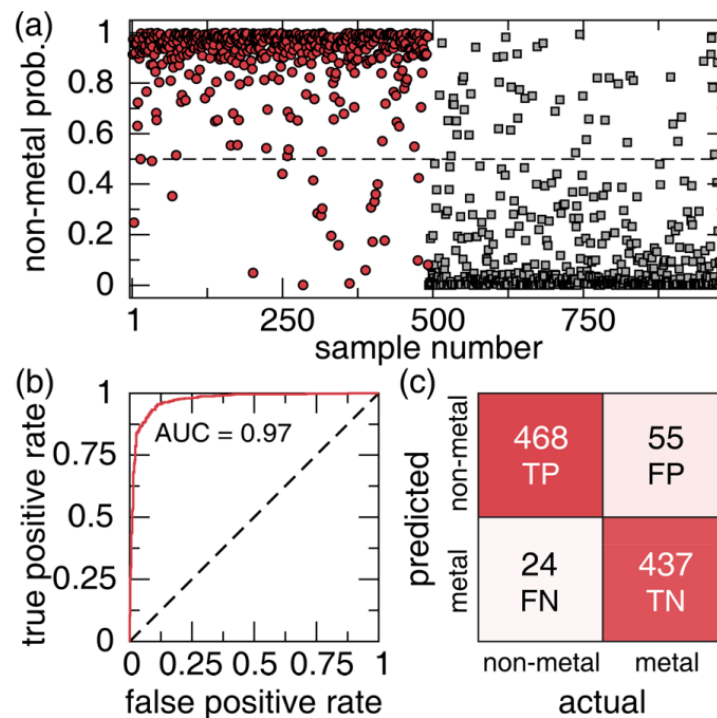
밴드갭 데이터셋을 활용한 금속 비금속 분류

J. Phys. Chem. Lett. 2018, 9, 7, 1668–1673

실험

TiCl	3.50		
TiSbS2	1.54		
Zn3As2	1.09		
Zn3As2	0.98		
ZnGeAs2	1.05		
ZnO	3.35		
ZnS	3.87		
ZnS	3.60		
ZnSe	2.67		
ZnSnP2	1.30		
ZnTe	2.39		
ZnTe	2.25		
ZnTe	2.29		
ZnSe	2.76		
ZnSnP2	1.66		
ZnTe	2.40		
AgCNO,65	0		
Al13Ru4	0		
SiTe2Os	0		
HoMg2	0		
UAl4	0		
SmCoC2	0		
Ag9Pb4O12	0		
NaZr2TiF11,12	0		
ScRh	0		
HoSi2Pt2	0		
Li3VOF5,1	0		
Ti2ZnS4	0		
KV6O11	0		
- - -	-		

계산



SimPL-ML www.simpl-ml.org

SimPL-ML

[Dataset Editor](#) [Model Generator](#) [Predictor](#) [Band-Gap Predictor](#) [Documentation](#) [Manage](#)  

Metal/Non-Metal Classification

Band gap (Binary) Prediction

Band gap (Ternary) Prediction

Band gap (Quaternary) Prediction

Classify Metal or Non-Metal.

Predict

SiO2

Metal or Non-metal : SiO2 is Non-metal !



Al

Metal or Non-metal : Al is Metal!



Au

Metal or Non-metal : Au is Metal!





SimPL-ML 플랫폼 요약

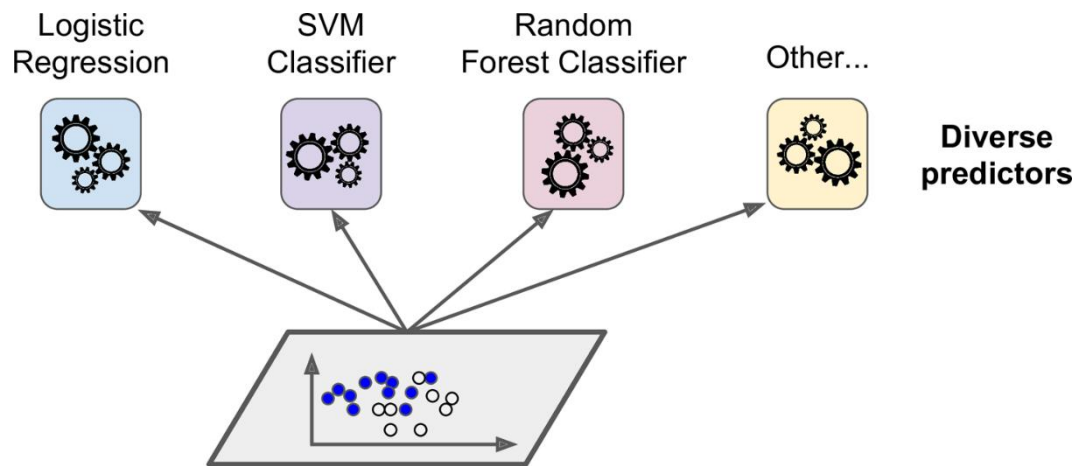
- ❖ 사용자의 Dataset을 활용하여 ML Model 생성 및 저장, 활용 용이.
- ❖ Atomic Feature Auto-Generation기능, Hyper Parameter Optimization기능, Feature/Target 선택기능 등을 통해 Feature engineering 및 이를 통한 예측 모델 최적화 가능.
- ❖ Binary, Ternary, Quaternary 무기소재에 대한 최적화된 밴드갭 예측모델 개발 및 플랫폼 탑재 완료.
- ❖ 사용자 요구에 맞춘 지속적 Dataset 및 예측모델 확장 가능.

An Easy, Simple, and Accessible Web-based Machine Learning Platform, SimPL-ML
Integrating Materials and Manufacturing Innovation 11, 85-94 (2022)
Seunghun Jang*, Gyoung S. Na, Jungho Lee, Jung Ho Shin, Hyun Woo Kim & Hyunju Chang*

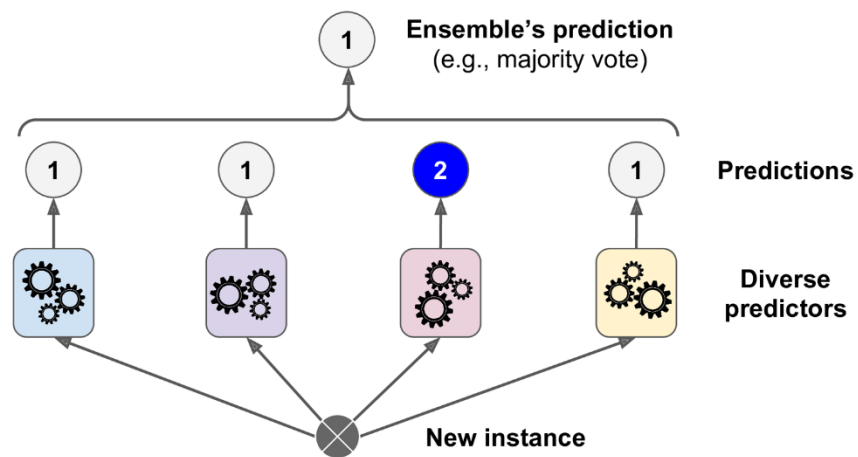
Regression

Ensemble , Decision Tree, Random Forest , GBTR

투표기반분류

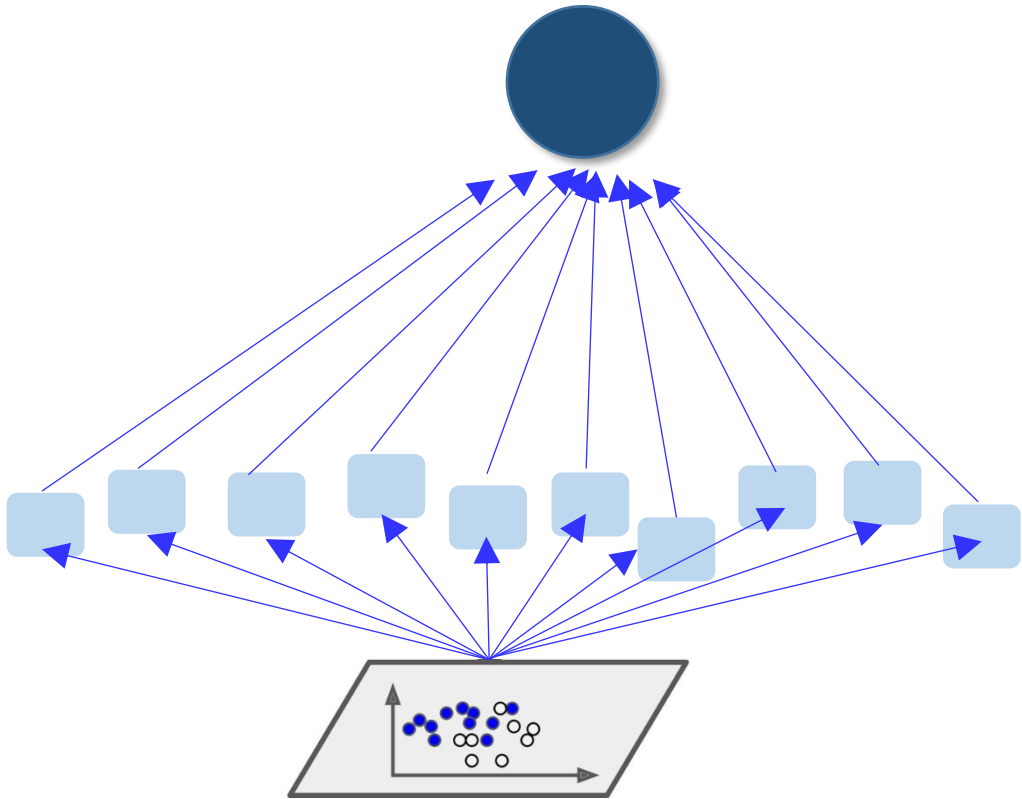


- 하나의 dataset에 대해서 여러 종류의 분류기를 적용.

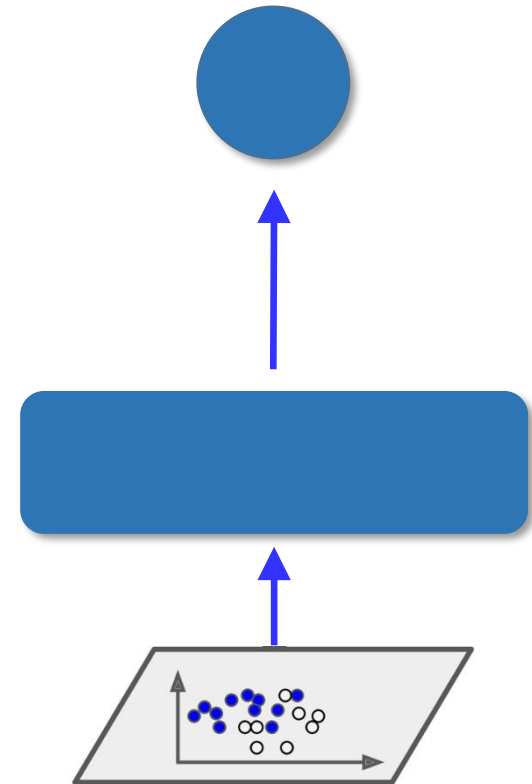


- 각 분류기의 예측결과중에서 가장 많이 선택된 클래스로 최종 예측결과를 정하는 방식. (다수결방식)

함께 하면 강력해지는 분류기.



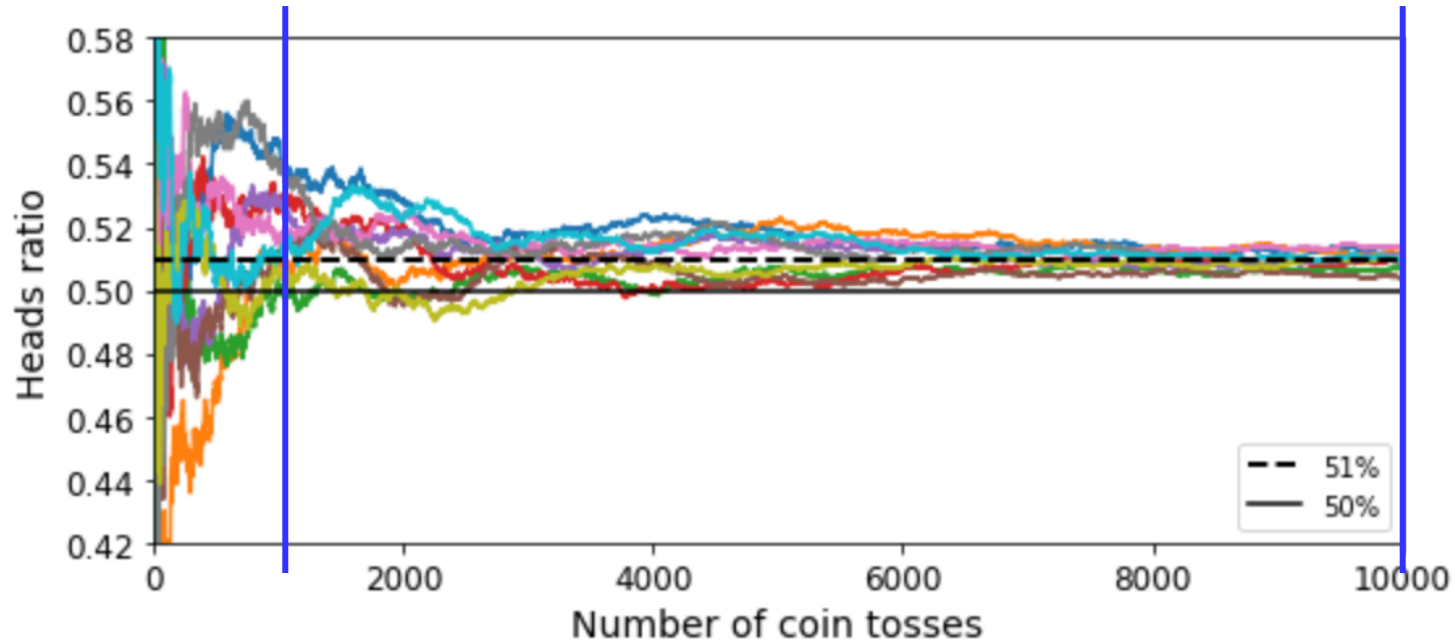
약한 학습기



강한 학습기

큰 수의 법칙

- 동전을 던졌을 때 앞면이 51%, 뒷면이 49%가 나오는 균형이 맞지 않는 동전이 있다고 가정.
- 수학적으로 계산해보면, 1000번 던진 후 앞면이 다수가 될 확률은 75%에 가까움
- 10000번을 던지면 확률이 거의 100%로 수렴.



- 균형이 틀어진 동전을 10번 실험한 결과
- 10번의 실험에서도 모두 많이 던질수록 앞면이 나올 확률은 51%로 수렴.

VotingClassifier

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

log_clf = LogisticRegression(solver="lbfgs", random_state=42)
rnd_clf = RandomForestClassifier(n_estimators=100, random_state=42)
svm_clf = SVC(gamma="scale", random_state=42)

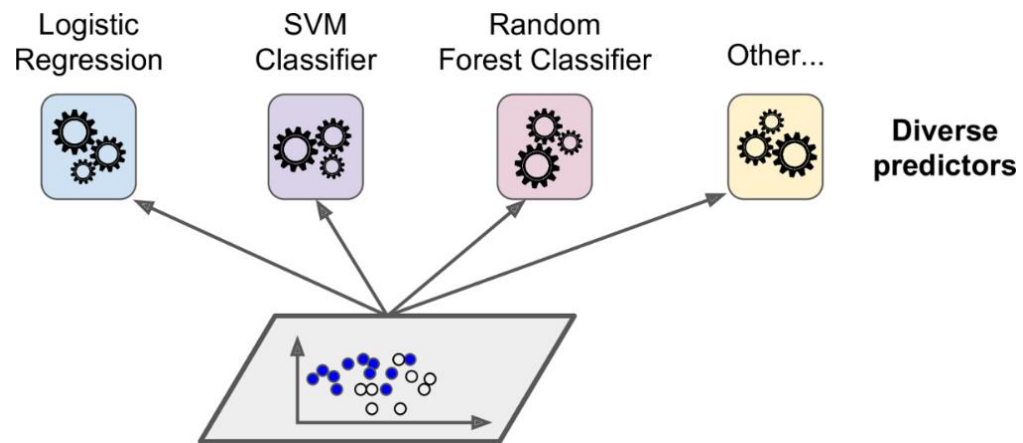
voting_clf = VotingClassifier(
    estimators=[('lr', log_clf), ('rf', rnd_clf), ('svc', svm_clf)],
    voting='hard')
```

```
from sklearn.metrics import accuracy_score

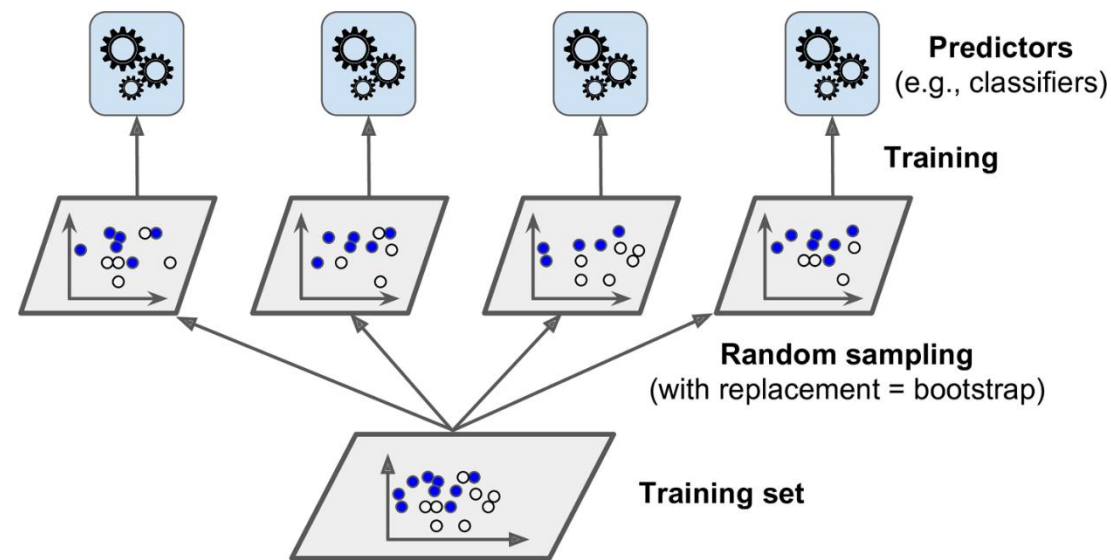
for clf in (log_clf, rnd_clf, svm_clf, voting_clf):
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    print(clf.__class__.__name__, accuracy_score(y_test, y_pred))
```

```
LogisticRegression 0.864
RandomForestClassifier 0.896
SVC 0.896
VotingClassifier 0.912
```

다양한 분류기 활용 방식



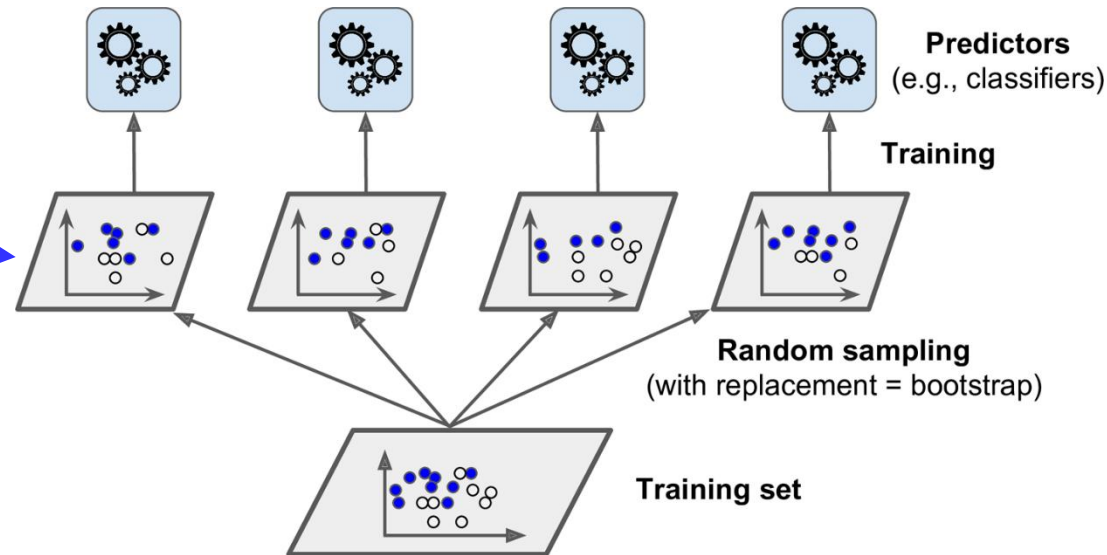
- 각각 다른 훈련 알고리즘을 사용하는 방식



- 같은 알고리즘을 사용하고 훈련세트의 서브셋을 무작위로 구성하여 여러 개의 분류기를 학습하는 방식

Bagging and Pasting

- Bagging : 훈련세트에서 중복을 허용하여 샘플링하는 방식(boots**tr**ap **agg**regating)
- Pasting : 중복을 허용하지 않고 샘플링을 하는 방식



- 분류(Classification) : 통계적 최빈값 (다수결 방식)
- 회귀(Regression) : 평균값

사이킷런 활용 Bagging 기반 분류

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

bag_clf = BaggingClassifier(
    DecisionTreeClassifier(), n_estimators=500,
    max_samples=100, bootstrap=True, random_state=42)
bag_clf.fit(X_train, y_train)
y_pred = bag_clf.predict(X_test)
```

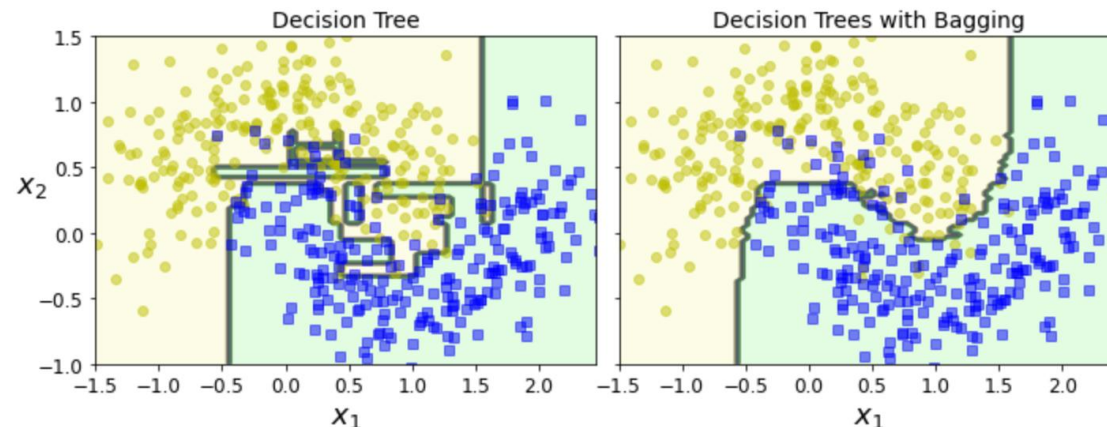
```
from sklearn.metrics import accuracy_score
print(accuracy_score(y_test, y_pred))
```

0.904 → Decision Tree with Bagging

```
tree_clf = DecisionTreeClassifier(random_state=42)
tree_clf.fit(X_train, y_train)
y_pred_tree = tree_clf.predict(X_test)
print(accuracy_score(y_test, y_pred_tree))
```

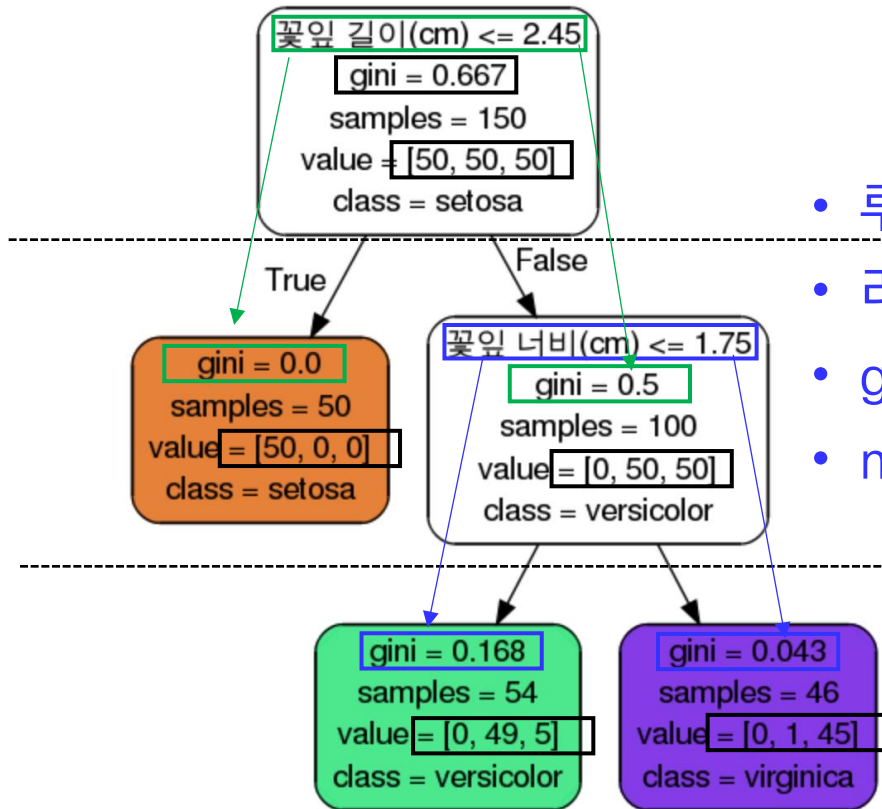
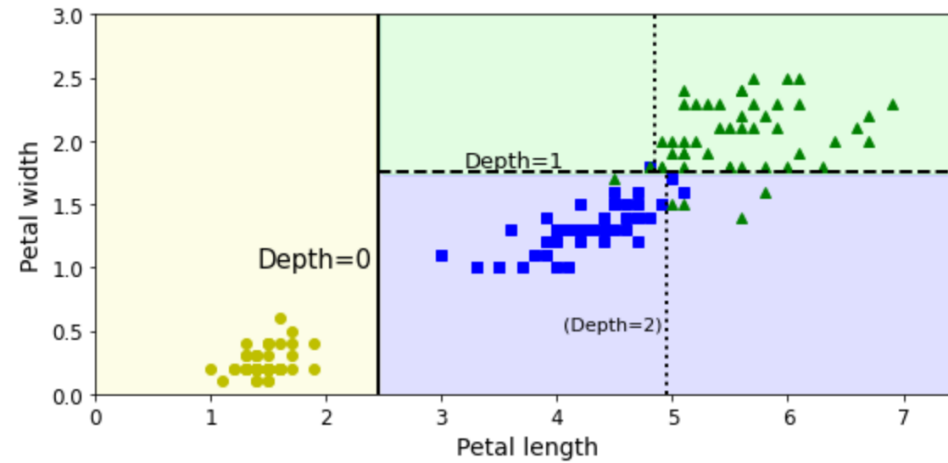
0.856 → Decision Tree

- 결정트리 분류기 500개의 앙상블 훈련
- 훈련세트에서 중복을 허용, 무작위로 선택된 100개의 샘플로 훈련.
- Pasting을 사용하려면, bootstrap = False



Decision Boundary 비교

Decision Tree



- 루트 노드 (깊이 0)
- 리프 노드 (깊이 1, 2)
- gini : 불순도
- max_depth (최대깊이)

순도(homogeneity)가 증가, 불순도(impurity) 혹은 불확실성(uncertainty)이 최대한 감소하도록 하는 방향으로 학습을 진행

gini = 0.168
samples = 54
value = [0, 49, 5]
class = versicolor

식 6-1: 지니 불순도

$$G_i = 1 - \sum_{k=1}^n p_{i,k}^2$$

$$\text{gini score} = 1 - (0/54)^2 - (49/54)^2 - (5/54)^2$$

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import DecisionTreeRegressor
```



Random Forest

- Bagging 방법을 적용한 결정 트리의 앙상블 학습 알고리즘.
- Bagging Classifier (DecisionTree Classifier) = Random Forest Classifier
- Classification & Regression 둘다 가능.

```
from sklearn.ensemble import RandomForestClassifier

rnd_clf = RandomForestClassifier(n_estimators=500, max_leaf_nodes=16, random_state=42)
rnd_clf.fit(X_train, y_train)

y_pred_rf = rnd_clf.predict(X_test)
```

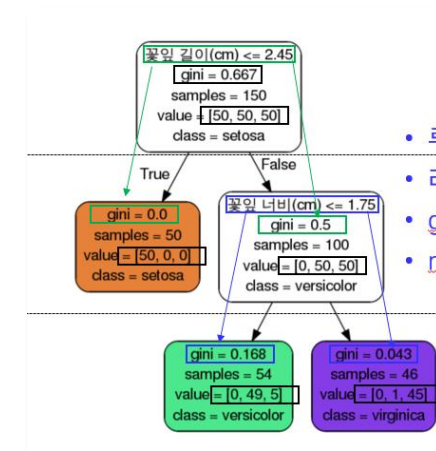
– 최대 16개의 리프노드를 갖는 500개의 트리로 이루어진 Random Forest 분류기.

Feature Importance

- 어떤 특성을 사용한 노드가 (Random Forest의 모든 트리에 걸쳐서) 평균적으로 불순도를 얼마나 감소시키는지 확인하여 특성의 중요도를 측정.

```
from sklearn.datasets import load_iris
iris = load_iris()
rnd_clf = RandomForestClassifier(n_estimators=500, random_state=42)
rnd_clf.fit(iris["data"], iris["target"])
for name, score in zip(iris["feature_names"], rnd_clf.feature_importances_):
    print(name, score)
```

꽃받침 sepal length (cm) 0.11249225099876375
sepal width (cm) 0.02311928828251033
꽃잎 petal length (cm) 0.4410304643639577
petal width (cm) 0.4233579963547682



Feature Importance



```
from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', version=1)
mnist.target = mnist.target.astype(np.uint8)

rnd_clf = RandomForestClassifier(n_estimators=100, random_state=42)
rnd_clf.fit(mnist["data"], mnist["target"])

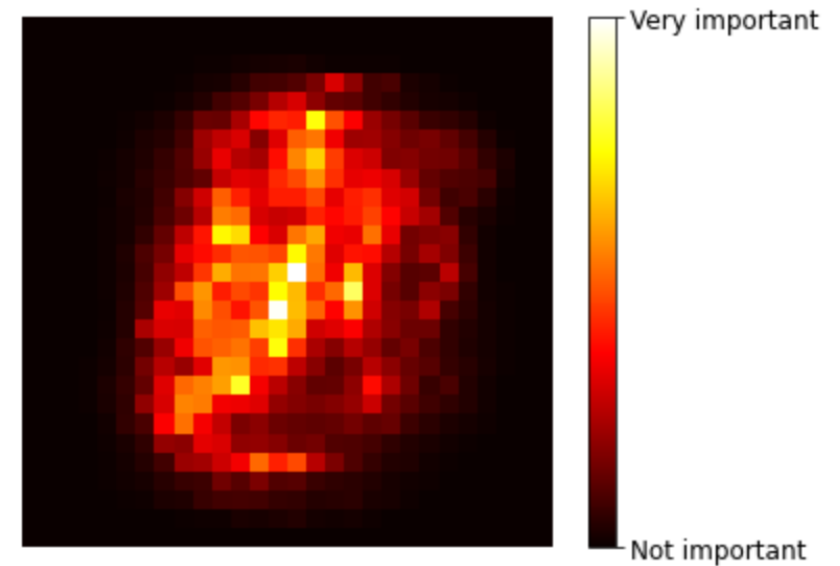
RandomForestClassifier(random_state=42)

def plot_digit(data):
    image = data.reshape(28, 28)
    plt.imshow(image, cmap = mpl.cm.hot,
               interpolation="nearest")
    plt.axis("off")

plot_digit(rnd_clf.feature_importances_)

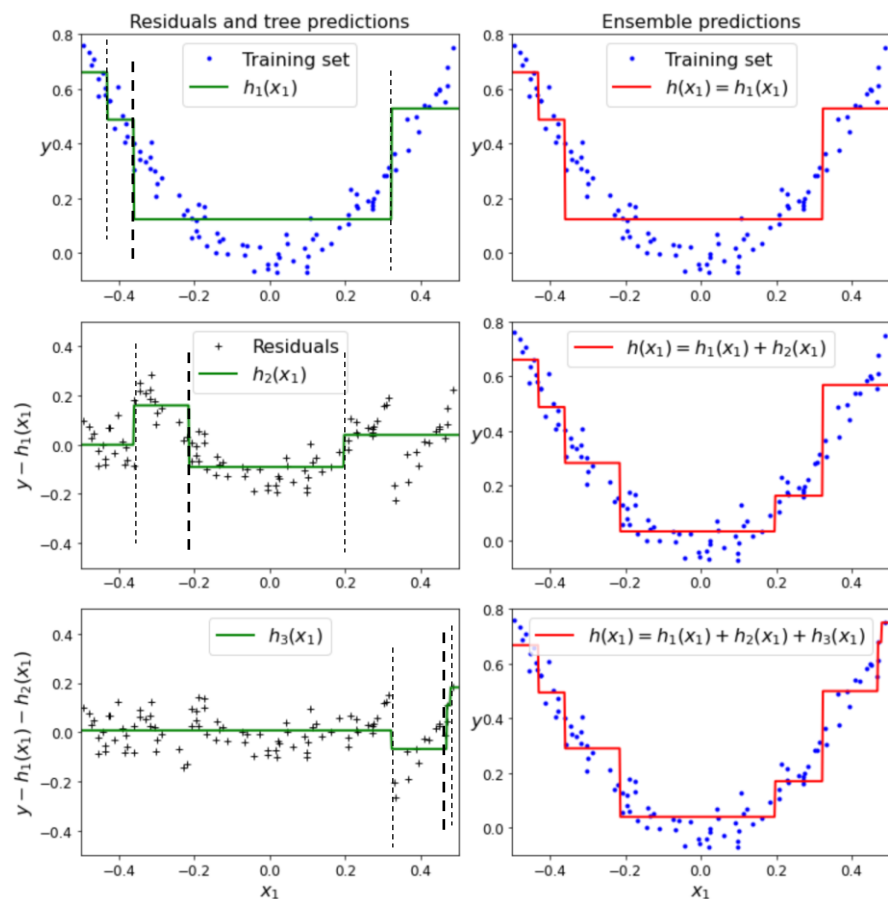
cbar = plt.colorbar(ticks=[rnd_clf.feature_importances_.min(), rnd_clf.feature_importances_.max()])
cbar.ax.set_yticklabels(['Not important', 'Very important'])

save_fig("mnist_feature_importance_plot")
plt.show()
```



Gradient Boosting Tree 방법

- 이전 예측기가 만든 잔여오차(residual error)에 새로운 예측기를 학습시키는 방법



```
from sklearn.ensemble import GradientBoostingRegressor
```

```
gbrt = GradientBoostingRegressor(max_depth=2, n_estimators=3, learning_rate=1.0, random_state=42)  
gbrt.fit(X, y)
```

MNIST Dataset을 활용한 분류 모델, Random Forest 실습

실습파일 다운로드 주소 및 QR 코드

https://github.com/SeunghunJang/202509_UST_Jang



202509_소재 정보학 강의_실습자료.zip 압축해제 후
jupyter notebook에서 열기.

실습 환경

- Google Colab
- K-MDS jupyterlab (<https://jupyter.kmds.re.kr/user/>)
 - !pip install scikit-learn
 - !pip install matplotlib
 - !pip install pandas

MNIST Dataset을 활용한 분류 모델, Random Forest 실습

MNIST Dataset을 활용한 분류 모델 실습

```
In [45]: # 파이썬 ≥3.5 필수, 이하일 경우 assert 에러
import sys
assert sys.version_info >= (3, 5)

# sklearn version 0.20 이상.
import sklearn
assert sklearn.__version__ >= "0.20"
print('The scikit-learn version is {}'.format(sklearn.__version__))

# 공통 모듈 임포트
import numpy as np
import os

# 실습 결과를 동일하게 하기 위해서
np.random.seed(42)

# 깔끔한 그래프 출력을 위해서
%matplotlib inline
import matplotlib as mpl
import matplotlib.pyplot as plt
mpl.rc('axes', labelsizе=14)
mpl.rc('xtick', labelsizе=12)
mpl.rc('ytick', labelsizе=12)

# 그림을 저장할 위치 설정
PROJECT_ROOT_DIR = "."
CHAPTER_ID = "classification"
IMAGES_PATH = os.path.join(PROJECT_ROOT_DIR, "images", CHAPTER_ID)
os.makedirs(IMAGES_PATH, exist_ok=True)

def save_fig(fig_id, tight_layout=True, fig_extension="png", resolution=300):
    path = os.path.join(IMAGES_PATH, fig_id + "." + fig_extension)
    print("그림 저장:", fig_id)
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)
```

MNIST Dataset을 활용한 분류 모델, Random Forest 실습

In [46]: #MNIST dataset 다운로드 받기 (약간의 시간 필요)

```
from sklearn.datasets import fetch_openml
mnist = fetch_openml('mnist_784', version=1, as_frame=False)
mnist.keys()
```

#MNIST dataset은 딕셔너리 구조를 가지고 있음.
#데이터셋을 설명하는 'DESCR' 키
#샘플이 하나의 행, 특성이 하나의 열로 구성된 배열을 가진 'data' 키

Out[46]: dict_keys(['data', 'target', 'frame', 'categories', 'feature_names', 'target_names', 'DESCR', 'details', 'url'])

In [49]:

mnist

Out[49]: {'data': array([[0., 0., 0., ..., 0., 0., 0.],
 [0., 0., 0., ..., 0., 0., 0.],
 [0., 0., 0., ..., 0., 0., 0.],
 ...,
 [0., 0., 0., ..., 0., 0., 0.],
 [0., 0., 0., ..., 0., 0., 0.],
 [0., 0., 0., ..., 0., 0., 0.])),
 'target': array(['5', '0', '4', ..., '4', '5', '6'], dtype=object),
 'frame': None,
 'categories': {},
 'feature_names': ['pixel1',
 'pixel2',
 'pixel3',
 'pixel4',
 'pixel5',
 'pixel6',
 'pixel7',
 'pixel8',
 'pixel9',
 'pixel10',
 'pixel11',
 'pixel12',
 'pixel13',
 'pixel14',
 'pixel15',
 'pixel16',
 'pixel17',
 'pixel18',
 'pixel19',
 'pixel20',
 'pixel21',
 'pixel22',
 'pixel23',
 'pixel24',
 'pixel25',
 'pixel26',
 'pixel27',
 'pixel28',
 'pixel29',
 'pixel30',
 'pixel31',
 'pixel32',
 'pixel33',
 'pixel34',
 'pixel35',
 'pixel36',
 'pixel37',
 'pixel38',
 'pixel39',
 'pixel40',
 'pixel41',
 'pixel42',
 'pixel43',
 'pixel44',
 'pixel45',
 'pixel46',
 'pixel47',
 'pixel48',
 'pixel49',
 'pixel50',
 'pixel51',
 'pixel52',
 'pixel53',
 'pixel54',
 'pixel55',
 'pixel56',
 'pixel57',
 'pixel58',
 'pixel59',
 'pixel60',
 'pixel61',
 'pixel62',
 'pixel63',
 'pixel64',
 'pixel65',
 'pixel66',
 'pixel67',
 'pixel68',
 'pixel69',
 'pixel70',
 'pixel71',
 'pixel72',
 'pixel73',
 'pixel74',
 'pixel75',
 'pixel76',
 'pixel77',
 'pixel78',
 'pixel79',
 'pixel80',
 'pixel81',
 'pixel82',
 'pixel83',
 'pixel84',
 'pixel85',
 'pixel86',
 'pixel87',
 'pixel88',
 'pixel89',
 'pixel90',
 'pixel91',
 'pixel92',
 'pixel93',
 'pixel94',
 'pixel95',
 'pixel96',
 'pixel97',
 'pixel98',
 'pixel99']], dtype=float64),
 'target_names': None,
 'DESCR': 'mnist_784',
 'details': 'mnist_784',
 'url': 'mnist_784'}

In [50]: X, y = mnist["data"], mnist["target"]
X.shape

Out[50]: (70000, 784)

MNIST Dataset을 활용한 분류 모델, Random Forest 실습

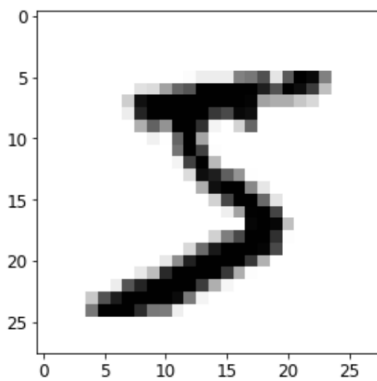
In [53]: # 첫번째 샘플에 대한 특성 data를 28,28 픽셀로 출력하기

```
import matplotlib as mpl
import matplotlib.pyplot as plt

some_digit = X[0]
some_digit_image = some_digit.reshape(28, 28)
plt.imshow(some_digit_image, cmap=mpl.cm.binary)
plt.axis("on")

save_fig("some_digit_plot")
plt.show()
```

그림 저장: some_digit_plot



In [54]: #첫번째 샘플의 레이블 확인하기
y[0]

Out[54]: '5'

In [55]: ## 레이블은 숫자가 아닌 문자열
머신러닝 알고리즘은 숫자를 기대하므로 y를 정수로 변환

```
y = y.astype(np.uint8)
```

In [56]: # 샘플의 특성 data를 28,28 픽셀로 출력하는 함수 만들기.

```
def plot_digit(data):
    image = data.reshape(28, 28)
    plt.imshow(image, cmap = mpl.cm.binary,
                interpolation="nearest")
    plt.axis("off")
```

MNIST Dataset을 활용한 분류 모델, Random Forest 실습

In [57]: # 숫자 그림을 위한 추가 함수

```
def plot_digits(instances, images_per_row=10, **options):
    size = 28
    images_per_row = min(len(instances), images_per_row)
    # n_rows = ceil(len(instances) / images_per_row) 와 동일합니다:
    n_rows = (len(instances) - 1) // images_per_row + 1

    # 필요하면 그리드 끝을 채우기 위해 빈 이미지를 추가합니다:
    n_empty = n_rows * images_per_row - len(instances)
    padded_instances = np.concatenate([instances, np.zeros((n_empty, size * size))], axis=0)

    # 배열의 크기를 바꾸어 28x28 이미지를 담은 그리드로 구성합니다:
    image_grid = padded_instances.reshape((n_rows, images_per_row, size, size))

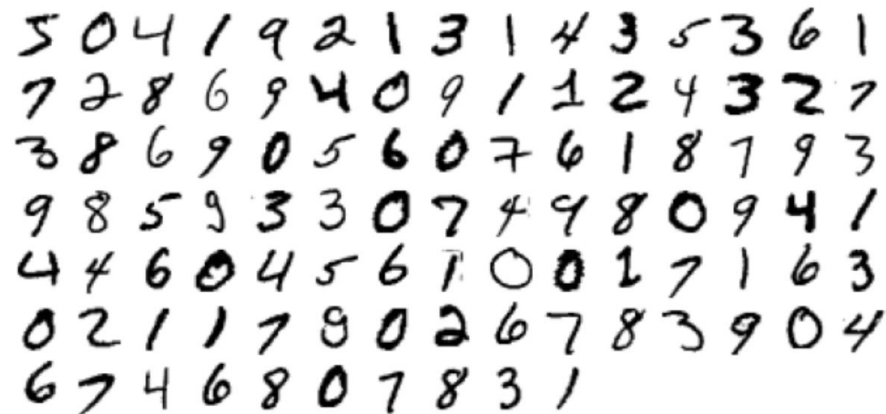
    # 축 0(이미지 그리드의 수직축)과 2(이미지의 수직축)를 합치고 축 1과 3(두 수평축)을 합칩니다.
    # 먼저 transpose()를 사용해 결합하려는 축을 옆으로 이동한 다음 합칩니다:
    big_image = image_grid.transpose(0, 2, 1, 3).reshape(n_rows * size,
                                                         images_per_row * size)

    # 하나의 큰 이미지를 얻었으므로 출력하면 됩니다:
    plt.imshow(big_image, cmap = mpl.cm.binary, **options)
    plt.axis("off")
```

In [58]: ## 10X10으로 그리려면 images_per_row를 10으로 변경.

```
plt.figure(figsize=(9,9))
example_images = X[:100]
plot_digits(example_images, images_per_row=15)
save_fig("more_digits_plot")
plt.show()
```

그림 저장: more_digits_plot



5 0 4 1 9 2 1 3 1 4 3 5 3 6 1
7 2 8 6 9 4 0 9 1 1 2 4 3 2 7
3 8 6 9 0 5 6 0 7 6 1 8 7 9 3
9 8 5 9 3 3 0 7 4 9 8 0 9 4 1
4 4 6 0 4 5 6 1 0 0 1 7 1 6 3
0 2 1 1 7 8 0 2 6 7 8 3 9 0 4
6 7 4 6 8 0 7 8 3 1

MNIST Dataset을 활용한 분류 모델, Random Forest 실습

Binary Classification 실습

In [64]: # 5는 True이고, 다른 숫자는 모두 False

```
y_train_5 = (y_train == 5)
y_test_5 = (y_test == 5)

#print (X_train)
#print (X_test.shape)
#print (y_train)
#print (y_test)
print (y_train_5[0], y_train_5[11], y_train_5[12])
print (y_test_5)
```

```
True True False
[False False False ... False  True False]
```

In [66]: # SGD Classifier 적용 분류 모델 만들어 보기.
5이면 true로, 5가 아니면 false로 분류하는 Binary Classification.

```
from sklearn.linear_model import SGDClassifier

sgd_clf = SGDClassifier(max_iter=1000, tol=1e-3, random_state=17)
sgd_clf.fit(X_train, y_train_5)
```

Out[66]: SGDClassifier(random_state=17)

In [67]: sgd_clf.predict([some_digit])

```
## some_digit = X[0]
```

Out[67]: array([True])

In [68]: ## cross_val_score : 단순교차검증
3-폴드 교차 검증 수행

```
from sklearn.model_selection import cross_val_score
cross_val_score(sgd_clf, X_train, y_train_5, cv=3, scoring="accuracy")
```

Out[68]: array([0.96675, 0.9618 , 0.94425])

MNIST Dataset을 활용한 분류 모델, Random Forest 실습

```
In [69]: ## StratifiedKFold : 계층별 k-겹 교차 검증 (클래스별 비율이 유지되도록 폴드를 만듦)
## k-fold 에서 데이터(레이블 or 클래스)가 한 곳으로 몰리는 것을 방지

from sklearn.model_selection import StratifiedKFold
from sklearn.base import clone

skfolds = StratifiedKFold(n_splits=3, random_state=42, shuffle=True)

## 위에서 만들 모델, "sgd_clf.fit(X_train, y_train_5)"을 "clone_clf"라는 이름으로 담기.

for train_index, test_index in skfolds.split(X_train, y_train_5):
    clone_clf = clone(sgd_clf)
    X_train_folds = X_train[train_index]
    y_train_folds = y_train_5[train_index]
    X_test_fold = X_train[test_index]
    y_test_fold = y_train_5[test_index]

    clone_clf.fit(X_train_folds, y_train_folds)
    y_pred = clone_clf.predict(X_test_fold)
    n_correct = sum(y_pred == y_test_fold)
    print(n_correct / len(y_pred))

0.9681
0.9651
0.9653
```

```
In [70]: ## 모든 이미지를 "5아님" 클래스로 분류하는 dummy classifier 생성해보기.
```

```
from sklearn.base import BaseEstimator
class Never5Classifier(BaseEstimator):
    def fit(self, X, y=None):
        pass
    def predict(self, X):
        return np.zeros((len(X), 1), dtype=bool)
```

```
In [74]: from sklearn.model_selection import cross_val_score

never_5_clf = Never5Classifier()
cross_val_score(never_5_clf, X_train, y_train_5, cv=3, scoring="accuracy")

## 10개중에 1개만 "5"이므로 모두 5가 아니라고 분류하는 모델을 만들면
## 정확도가 90% 이상 나오게 됨.
## Accuracy가 높다고 해서 무조건 좋은 모델이라고 오판해서는 안됨!!
```

```
Out[74]: array([0.91125, 0.90855, 0.90915])
```

MNIST Dataset을 활용한 분류 모델, Random Forest 실습

In [77]: # 모두 False로 예측 하는지 체크!

```
from sklearn.model_selection import cross_val_predict
y_never_5_pred = cross_val_predict(never_5_clf, X_train, y_train_5, cv=3)
print(y_never_5_pred)
```

```
[[False]
 [False]
 [False]
 ...
 [False]
 [False]
 [False]]
```

In [78]: ## 다시 sgd_clf 모델(In 68에서 생성)을 활용하여 예측값에 대한 array (y_train_pred) 생성.

```
from sklearn.model_selection import cross_val_predict
y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3)
print(y_train_pred)
```

```
[ True False False ...  True False False]
```

In [80]: ## 오차 행렬 확인하기

```
from sklearn.metrics import confusion_matrix
confusion_matrix(y_train_5, y_train_pred)
```

Out[80]: array([[53936, 643],
 [1901, 3520]], dtype=int64)

In [26]: ## 예측값의 자리에 정답을 놓고 오차 행렬을 확인해보면...
완벽한 분류기의 결과를 보임.

```
y_train_perfect_predictions = y_train_5
confusion_matrix(y_train_5, y_train_perfect_predictions)
```

Out[26]: array([[54579, 0],
 [0, 5421]], dtype=int64)

MNIST Dataset을 활용한 분류 모델, Random Forest 실습

```
In [81]: ## 정밀도 확인 (sklearn 함수 제공)

from sklearn.metrics import precision_score, recall_score

precision_score(y_train_5, y_train_pred)
```

Out[81]: 0.8455440787893346

```
In [82]: ## 정밀도를 CM으로부터 직접 계산

cm = confusion_matrix(y_train_5, y_train_pred)
cm[1, 1] / (cm[0, 1] + cm[1, 1])
```

Out[82]: 0.8455440787893346

```
In [83]: ## 재현율 확인 (sklearn 함수 제공)

recall_score(y_train_5, y_train_pred)
```

Out[83]: 0.6493266924921601

```
In [84]: cm[1, 1] / (cm[1, 0] + cm[1, 1])
```

Out[84]: 0.6493266924921601

```
In [85]: ## F1 score 확인 (sklearn 함수 제공)

from sklearn.metrics import f1_score

f1_score(y_train_5, y_train_pred)
```

Out[85]: 0.7345575959933222

```
In [86]: cm[1, 1] / (cm[1, 1] + (cm[1, 0] + cm[0, 1]) / 2)
```

Out[86]: 0.7345575959933222

MNIST Dataset을 활용한 분류 모델, Random Forest 실습

정밀도/재현율 트레이드오프

```
In [93]: ## decision_function()메서드를 호출하여 각 샘플의 점수를 확인해보자.  
## 이 점수가 임계값(threshold)보다 크면 양성(True)으로 분류함.
```

```
y_scores = sgd_clf.decision_function([some_digit])  
y_scores
```

```
Out[93]: array([1067.64798936])
```

```
In [94]: threshold = 0  
y_some_digit_pred = (y_scores > threshold)
```

```
In [95]: y_some_digit_pred
```

```
Out[95]: array([ True])
```

```
In [96]: threshold = 8000  
y_some_digit_pred = (y_scores > threshold)  
y_some_digit_pred
```

```
## 이미지가 실제로 숫자 5이고 임계값이 0일때는 분류기가 이를 감지했지만, 임계값을 높이면 이를 놓치게 됨.  
## 따라서 True Positive는 줄어들고 False Negative은 늘어나므로 재현율은 감소하게 됨.  
## Threshold가 양의 방향으로 커지면 재현율은 감소하게 됨.
```

```
Out[96]: array([False])
```

```
In [97]: ## decision_function()메서드를 사용하여 y_scores array 생성.  
  
y_scores = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3,  
                             method="decision_function")  
y_scores
```

```
Out[97]: array([ 2402.52975074, -13366.16421546, -25713.60673974, ...,  
                4283.08132713, -13004.21253155, -21784.1670382 ])
```

MNIST Dataset을 활용한 분류 모델, Random Forest 실습

In [98]: `## precision_recall_curve` 함수를 사용하면 `threshold` 값에 따른 `precision`, `recall` 값들을 가져옴.

```
from sklearn.metrics import precision_recall_curve

precisions, recalls, thresholds = precision_recall_curve(y_train_5, y_scores)

# 각 값들을 직접 확인해보자.
#print (precision_recall_curve(y_train_5, y_scores))
#print (precisions)
#print (recalls)
#print (thresholds)
```

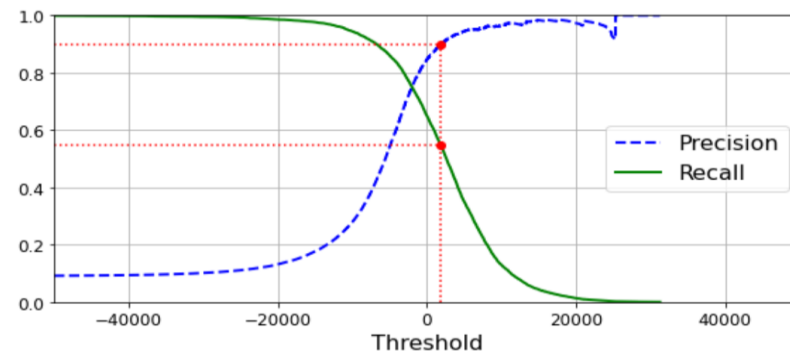
In [100]: `##threshold`에 따른 정밀도/재현율 그리기 함수 만들기.

```
def plot_precision_recall_vs_threshold(precisions, recalls, thresholds):
    plt.plot(thresholds, precisions[:-1], "b--", label="Precision", linewidth=2)
    plt.plot(thresholds, recalls[:-1], "g-", label="Recall", linewidth=2)
    plt.legend(loc="center right", fontsize=16) # Not shown in the book
    plt.xlabel("Threshold", fontsize=16)      # Not shown
    plt.grid(True)                           # Not shown
    plt.axis([-50000, 50000, 0, 1])          # Not shown

## 그리기 설정.
recall_90_precision = recalls[np.argmax(precisions >= 0.90)]
threshold_90_precision = thresholds[np.argmax(precisions >= 0.90)]

plt.figure(figsize=(8, 4))
plot_precision_recall_vs_threshold(precisions, recalls, thresholds)
plt.plot([threshold_90_precision, threshold_90_precision], [0., 0.9], "r:")
plt.plot([-50000, threshold_90_precision], [0.9, 0.9], "r:")
plt.plot([-50000, threshold_90_precision], [recall_90_precision, recall_90_precision], "r:")
plt.plot([threshold_90_precision], [0.9], "ro")
plt.plot([threshold_90_precision], [recall_90_precision], "ro")
save_fig("precision_recall_vs_threshold_plot")
plt.show()
```

그림 저장: `precision_recall_vs_threshold_plot`



MNIST Dataset을 활용한 분류 모델, Random Forest 실습

```
In [101]: ##(y_train_pred == (y_scores > 0)).all()
```

```
Out[101]: True
```

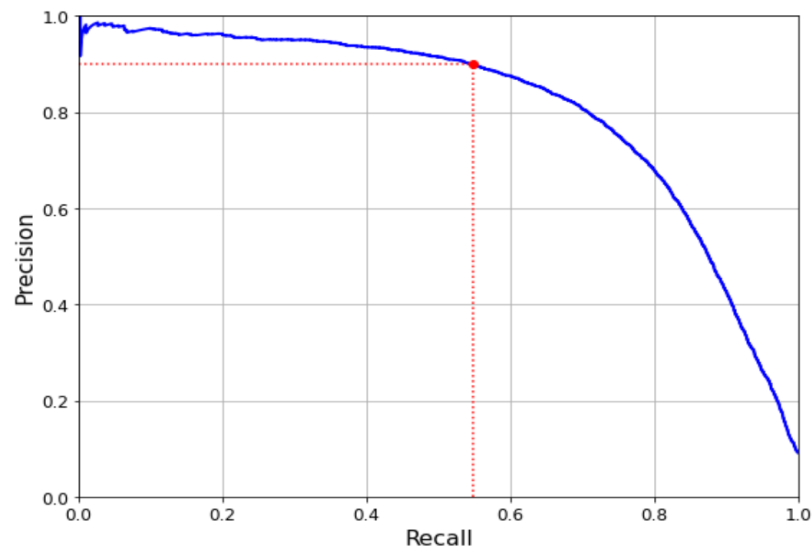
```
In [102]: ## 정밀도/재현율 trade off 관계 그림 그려보기.
```

```
print (precisions)
print (recalls)
```

```
def plot_precision_vs_recall(precisions, recalls):
    plt.plot(recalls, precisions, "b-", linewidth=2)
    plt.xlabel("Recall", fontsize=16)
    plt.ylabel("Precision", fontsize=16)
    plt.axis([0, 1, 0, 1])
    plt.grid(True)
```

```
plt.figure(figsize=(8, 6))
plot_precision_vs_recall(precisions, recalls)
plt.plot([recall_90_precision, recall_90_precision], [0., 0.9], "r:")
plt.plot([0.0, recall_90_precision], [0.9, 0.9], "r:")
plt.plot([recall_90_precision], [0.9], "ro")
save_fig("precision_vs_recall_plot")
plt.show()
```

```
[0.09044799 0.09043281 0.09043432 ... 1. 1. 1. ]
[1.00000000e+00 9.99815532e-01 9.99815532e-01 ... 3.68935621e-04
 1.84467810e-04 0.00000000e+00]
그림 저장: precision_vs_recall_plot
```



MNIST Dataset을 활용한 분류 모델, Random Forest 실습

```
In [103]: ## 정밀도 90% 달성을 목표로 설정.
```

```
threshold_90_precision = thresholds[np.argmax(precisions >= 0.90)]
```

```
In [104]: threshold_90_precision
```

```
Out[104]: 1854.5025990559493
```

```
In [106]: ## y_scores가 1854보다 크거나 같으면 True, 작으면 false 라고 하고,  
## 이 결과를 y_train_pred_90에 담기.
```

```
y_train_pred_90 = (y_scores >= threshold_90_precision)  
y_train_pred_90
```

```
Out[106]: array([ True, False, False, ...,  True, False, False])
```

```
In [108]: ## 실제 정확도를 확인해보기
```

```
precision_score(y_train_5, y_train_pred_90)
```

```
Out[108]: 0.9001214329083181
```

```
In [109]: ## 재현율도 확인해보기
```

```
recall_score(y_train_5, y_train_pred_90)
```

```
Out[109]: 0.5469470577384247
```

```
In [110]: ## ROC곡선 그려보기  
## roc_curve 함수를 통해서 threshold의 변화에 따른 fpr, tpr 값의 변화를 가져오기.
```

```
from sklearn.metrics import roc_curve
```

```
fpr, tpr, thresholds = roc_curve(y_train_5, y_scores)
```

MNIST Dataset을 활용한 분류 모델, Random Forest 실습

In [111]: ## ROC곡선 그리기 함수 만들기

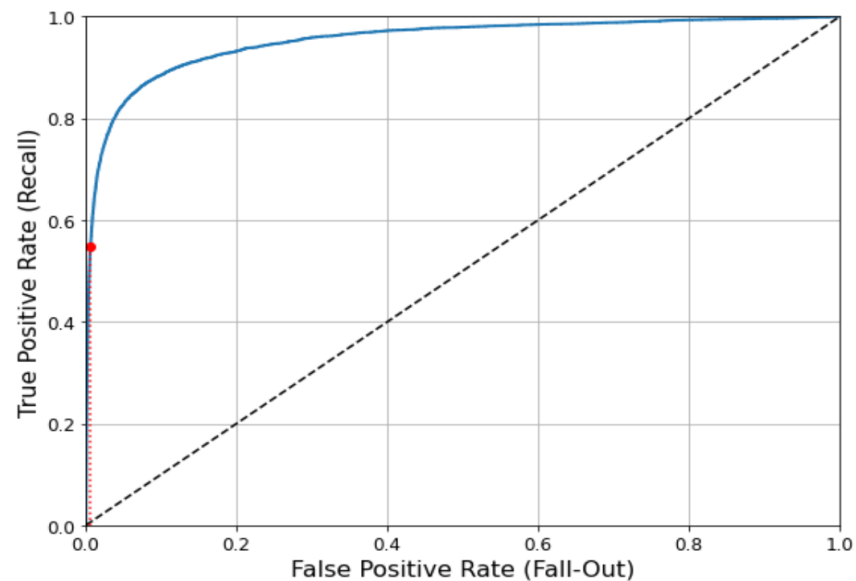
```
def plot_roc_curve(fpr, tpr, label=None):
    plt.plot(fpr, tpr, linewidth=2, label=label)
    plt.plot([0, 1], [0, 1], 'k--') # 대각 점선
    plt.axis([0, 1, 0, 1]) # Not shown in the book
    plt.xlabel('False Positive Rate (Fall-Out)', fontsize=16) # Not shown
    plt.ylabel('True Positive Rate (Recall)', fontsize=16) # Not shown
    plt.grid(True) # Not shown

plt.figure(figsize=(8, 6)) # Not shown
plot_roc_curve(fpr, tpr)
fpr_90 = fpr[np.argmax(tpr >= recall_90_precision)] # Not shown
print(fpr_90)
print(recall_90_precision)
# plt.plot([0.2, 0.7], [0.3, 0.5], "r:") # Not shown
plt.plot([fpr_90, fpr_90], [0., recall_90_precision], "r:") # Not shown
plt.plot([0.0, fpr_90], [recall_90_precision, recall_90_precision], "r:") # Not shown
plt.plot([fpr_90, [recall_90_precision], "ro") # Not shown
save_fig("roc_curve_plot") # Not shown
plt.show()
```

0.0060279594715916375

0.5469470577384247

그림 저장: roc_curve_plot



MNIST Dataset을 활용한 분류 모델, Random Forest 실습

In [117]: *## roc_auc_score 함수를 이용하여 분류기 성능 체크*

```
from sklearn.metrics import roc_auc_score

print (y_train_5)
print (y_scores)

roc_auc_score(y_train_5, y_scores)
```

```
[ True False False ...  True False False]
[ 2402.52975074 -13366.16421546 -25713.60673974 ...  4283.08132713
 -13004.21253155 -21784.1670382 ]
```

Out[117]: 0.953268587325405

In [115]: *## Random Forest 분류기로 훈련 시키기.*

```
from sklearn.ensemble import RandomForestClassifier
forest_clf = RandomForestClassifier(n_estimators=100, random_state=42)
y_probas_forest = cross_val_predict(forest_clf, X_train, y_train_5, cv=3,
                                    method="predict_proba")
```

In [119]: *## predict_proba 메서드를 활용하여 양성 클래스일 확률을 점수로 사용.*

```
y_scores_forest = y_probas_forest[:, 1] # 점수 = 양성 클래스의 확률

print (y_train_5)
print (y_scores_forest)

fpr_forest, tpr_forest, thresholds_forest = roc_curve(y_train_5, y_scores_forest)

[ True False False ...  True False False]
[0.89 0.01 0.04 ... 0.98 0.08 0.06]
```

MNIST Dataset을 활용한 분류 모델, Random Forest 실습

In [120]: *## SGD와 random forest 알고리즘에 따른 ROC 커브 비교.*

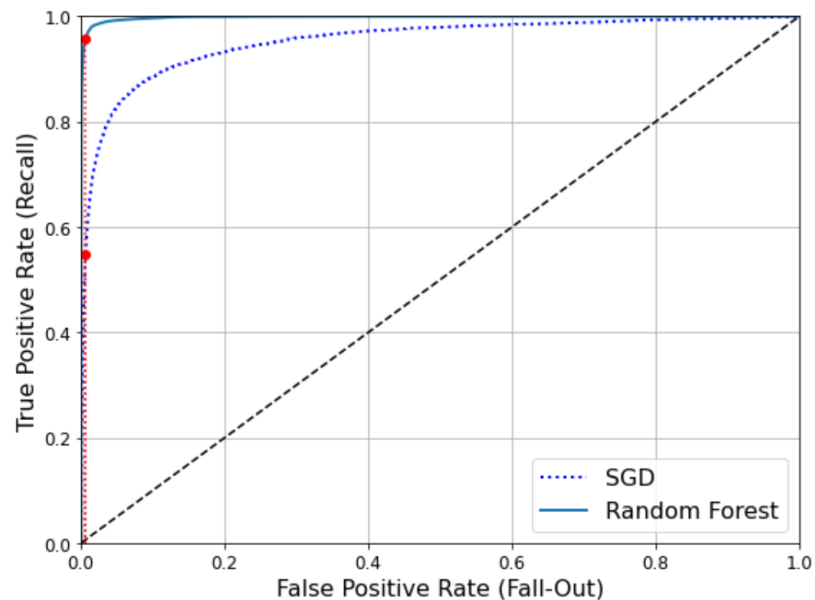
```
recall_for_forest = tpr_forest[np.argmax(fpr_forest >= fpr_90)]

print (recall_for_forest)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, "b:", linewidth=2, label="SGD")
plot_roc_curve(fpr_forest, tpr_forest, "Random Forest")
plt.plot([fpr_90, fpr_90], [0., recall_90_precision], "r:")
plt.plot([0.0, fpr_90], [recall_90_precision, recall_90_precision], "r:")
plt.plot([fpr_90], [recall_90_precision], "ro")
plt.plot([fpr_90, fpr_90], [0., recall_for_forest], "r:")
plt.plot([fpr_90], [recall_for_forest], "ro")
plt.grid(True)
plt.legend(loc="lower right", fontsize=16)
save_fig("roc_curve_comparison_plot")
plt.show()
```

0.9584947426674045

그림 저장: roc_curve_comparison_plot



* 실습

MNIST Dataset을 활용한 분류 모델, Random Forest 실습

```
In [121]: roc_auc_score(y_train_5, y_scores_forest)
```

```
Out[121]: 0.9983436731328145
```

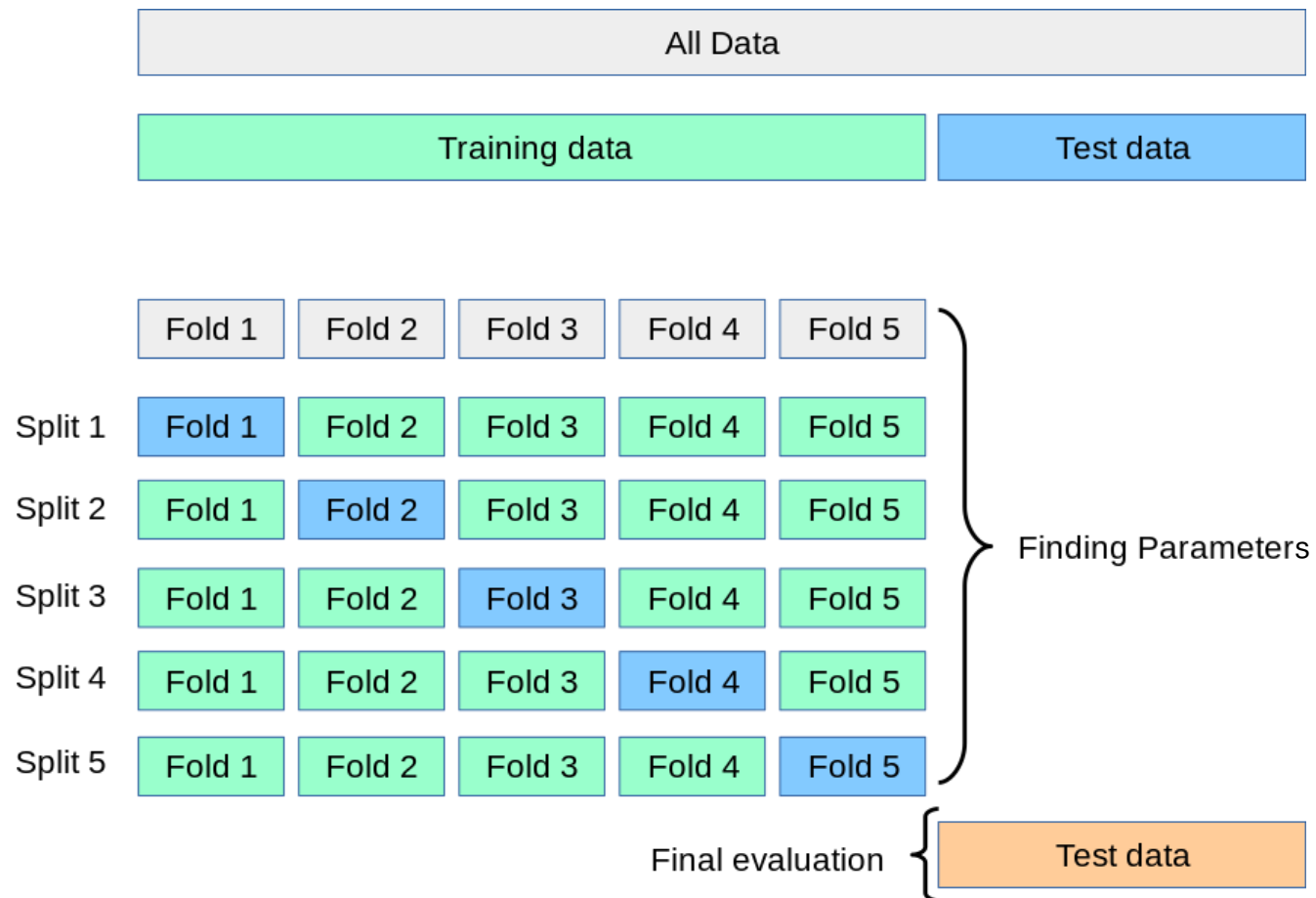
```
In [122]: y_train_pred_forest = cross_val_predict(forest_clf, X_train, y_train_5, cv=3)  
precision_score(y_train_5, y_train_pred_forest)
```

```
Out[122]: 0.9905083315756169
```

```
In [123]: recall_score(y_train_5, y_train_pred_forest)
```

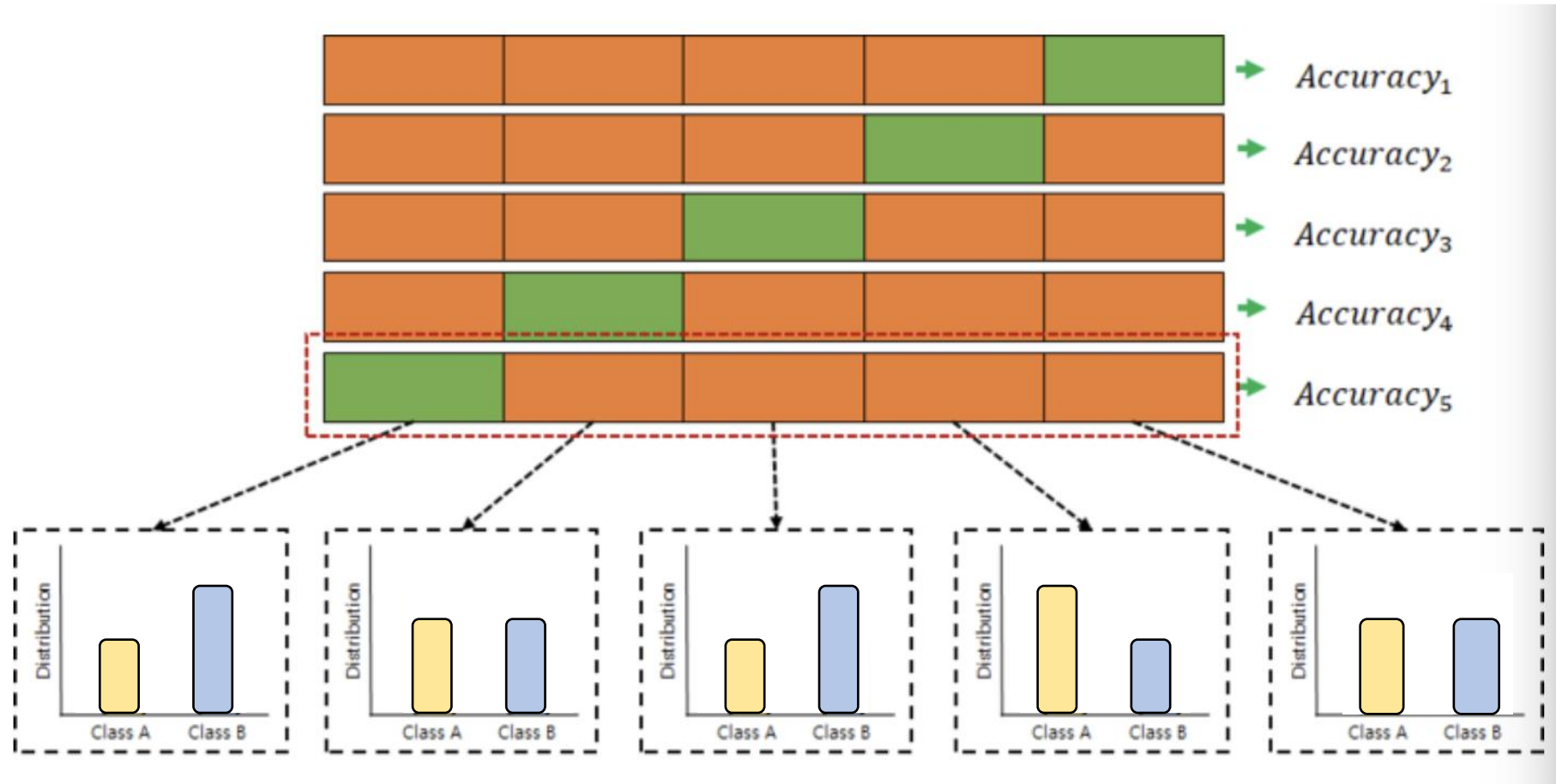
```
Out[123]: 0.8662608374838591
```

K겹 교차 검증



*이미지 출처 : <https://velog.io/@recoder/Cross-Validation>

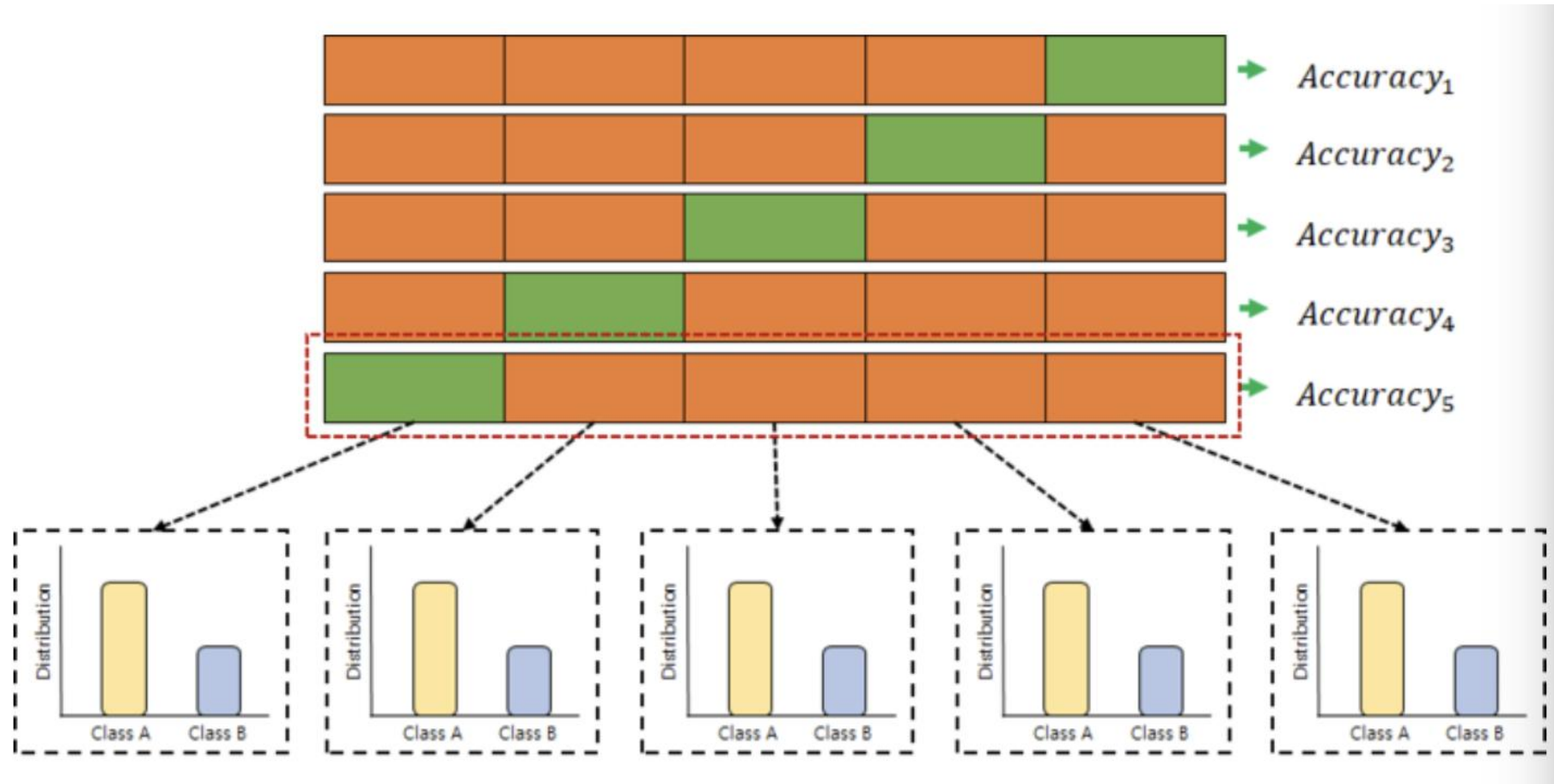
K겹 교차 검증의 문제점



*이미지 출처 : <https://huidea.tistory.com/30>

계층별 K겹 교차 검증

Stratified k-fold cross validation



*이미지 출처 : <https://huidea.tistory.com/30>

기초수학-선형대수-직교 행렬

- 직교 행렬(orthogonal matrix)이란 어떤 행렬의 행벡터(row vector)와 열벡터(column vector)가 유클리드 공간의 정규 직교 기저를 이루는 행렬을 의미
- 여기서 정규란 벡터의 길이가 1인 벡터를 말함.
- 직교라는 말은 벡터 간의 사이 각도가 90도임을 의미 (두 벡터의 내적이 0)
- 직교 행렬이란 행렬을 구성하는 각 행벡터 혹은 열벡터의 길이가 1이며 서로 수직인 벡터로 이루어진 행렬을 의미

$$AA^T = A^T A = I$$

- 위 식의 의미는 직교 행렬 A의 역행렬은 자기 자신의 전치 행렬(transposed matrix)임을 의미하며 $A^T=A^{-1}$ 이라고 쓸 수 있음.
- 이러한 성질을 이용하면 특정 행렬이 직교 행렬일때 역행렬을 구하기 쉬워짐.
- 일반적으로 행렬의 역행렬을 계산하는 방법은 복잡하지만,
- 직교행렬일 경우는 전치 행렬이 곧 역행렬이므로 더욱 쉽게 역행렬을 구할 수 있음.

직교행렬의 성질

- (1) 직교 행렬의 전치 행렬은 직교 행렬
- (2) 직교 행렬의 역행렬은 직교행렬
- (3) 직교 행렬끼리의 곱의 결과는 직교 행렬
- (4) 직교 행렬의 행렬식은 1 또는 -1

기초수학-선형대수-랭크, 차원

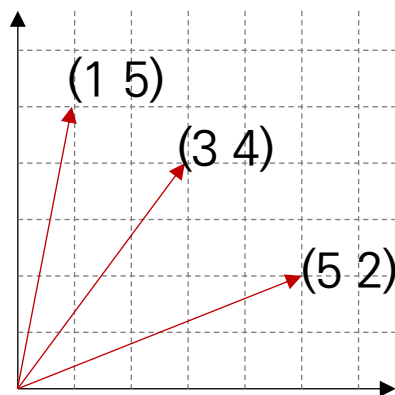
- **차원 (dimension)**이란 기저 벡터의 개수를 의미. 즉, 벡터 공간을 구성하는데 필요한 최소한의 벡터 개수가 차원. 예를 들어 **3차원 공간을 나타내는데 필요한 벡터 개수는 3개**, 그리고 행공간과 열공간의 차원은 같음.
- 랭크(rank)란 열벡터에 의해 span된 벡터공간의 차원 (아래 그림에서 2차원),

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{pmatrix}$$

행벡터

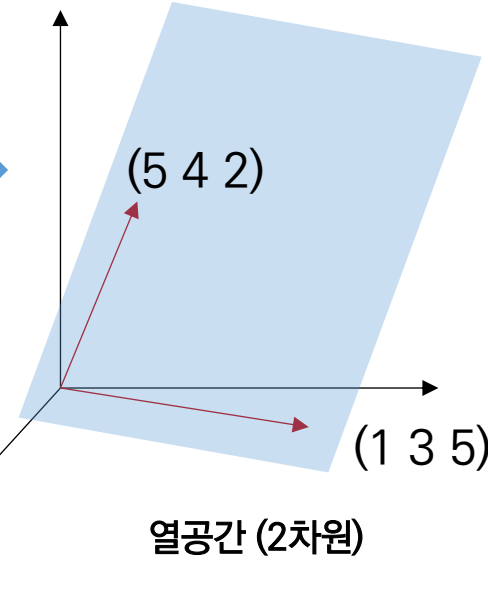
$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{pmatrix}$$

열벡터



행공간 (2차원)

$$\leftarrow A = \begin{pmatrix} 1 & 5 \\ 3 & 4 \\ 5 & 2 \end{pmatrix} \rightarrow$$



열공간 (2차원)

Thank you for your attention.
Please feel free to ask questions.

www.ai.chemdx.org
www.simpl-ml.org

한국화학연구원 디지털화학연구센터
장승훈, jang@kRICT.re.kr